

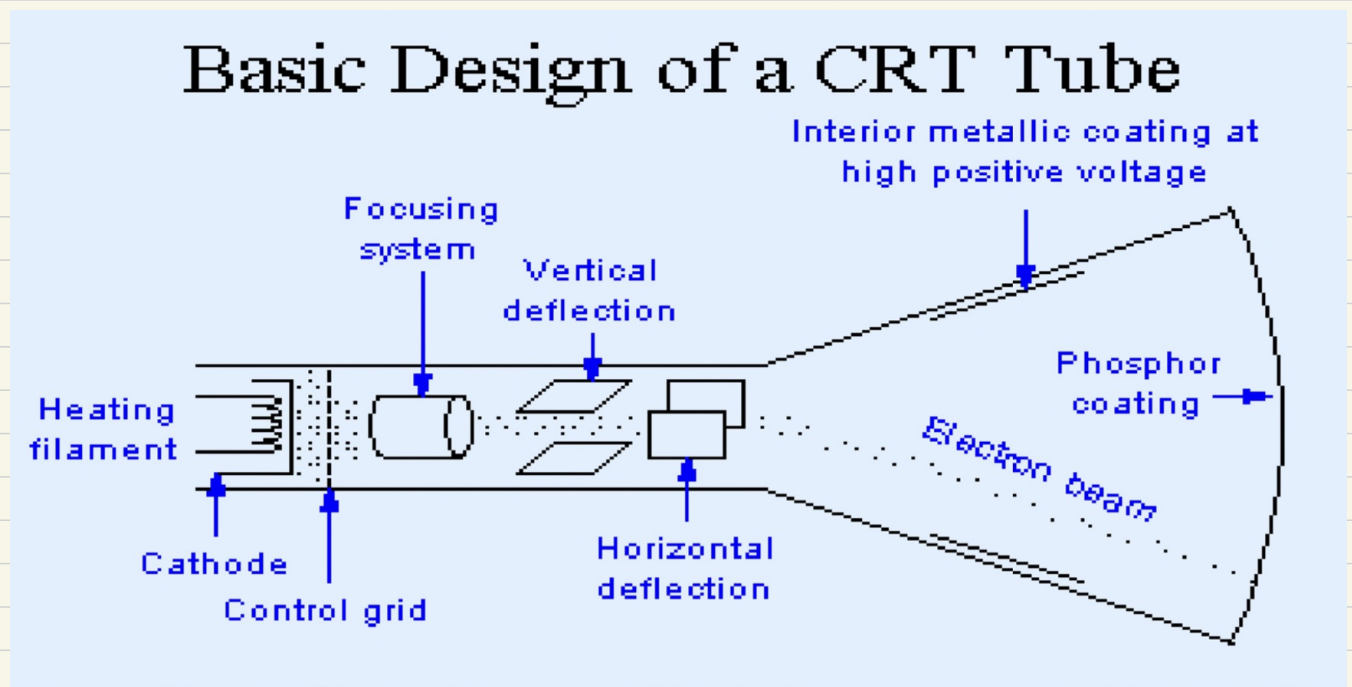
Unit 1 :->

-> Display Devices :->

Display devices are electronic devices that present visual information to the user. These devices include screens, monitors, projectors, and other technologies that can display text, images, videos, and other forms of content. They are used in a wide range of applications, from personal devices like smartphones and laptops to larger displays used in TVs, digital signage, and more. Display devices can use various technologies such as LCD (liquid crystal display), OLED (organic light-emitting diode), LED (light-emitting diode), and more to produce images and videos.

-> Cathode Ray Tube :->

A cathode ray tube (CRT) is a type of display technology that was widely used in older televisions and computer monitors. Its basic design consists of several key components:



Heating Filament : The CRT begins with a heating filament. When heated, cathode emits electrons through a process called thermionic emission.

Electron Gun (cathode): The electron gun emits a focused beam of electrons. This beam forms the basis for creating images on the screen.

Control Grid: It controls the intensity of the electron beam, allowing for modulation and control of the brightness of different parts of the screen.

Focusing System: To ensure that the electron beam remains focused, a focusing system uses electromagnetic fields to concentrate the electrons into a tight, narrow beam.

Horizontal and Vertical Deflection Systems: The electron beam is manipulated by horizontal and vertical deflection systems, which use electromagnetic fields to guide the beam across the screen in precise patterns. This scanning creates the visual image.

Phosphor-Coated Screen: The interior surface of the CRT is coated with phosphor, a material that emits visible light when struck by electrons. This is the surface on which the images are formed.

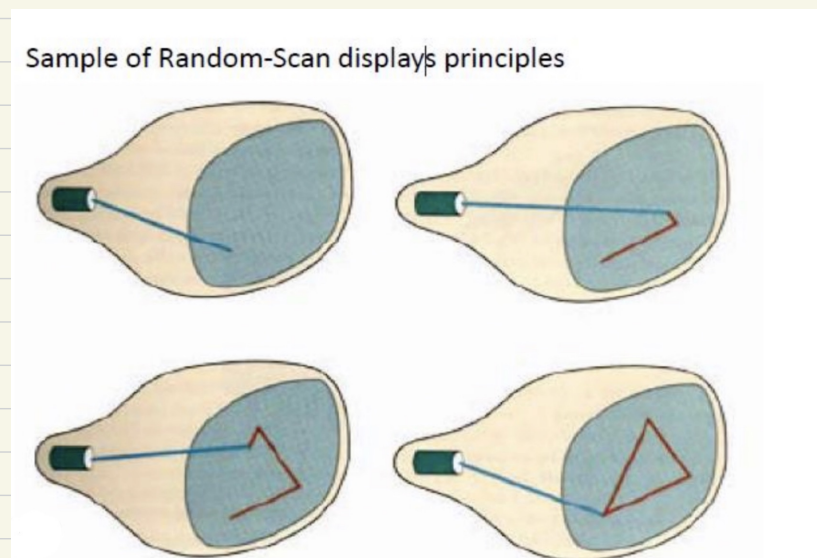
Interior Metallic Coating at High Positive Voltage (Aquadag Coating): To help maintain a uniform electric field within the tube, an interior metallic coating at a high positive voltage, often referred to as an aquadag coating, is applied to the inside of the CRT's glass surface.

Electron Beam: The focused beam of electrons emitted from the cathode travels through the control grid and is accelerated towards the phosphor-coated screen. When the electrons strike the phosphor, they cause it to emit light, creating the visible image.

Two techniques used for producing images on CRT displays are vector scan (or random scan) displays and raster scan displays.

① Vector Scan (Random Scan) Displays:

In vector scan displays, also known as random scan displays, images are created by directly drawing lines and shapes using an electron beam that moves along specified paths. These paths are defined by the coordinates of the endpoints of the lines or curves to be drawn. This technique is commonly used for computer-aided design (CAD) and graphics applications.



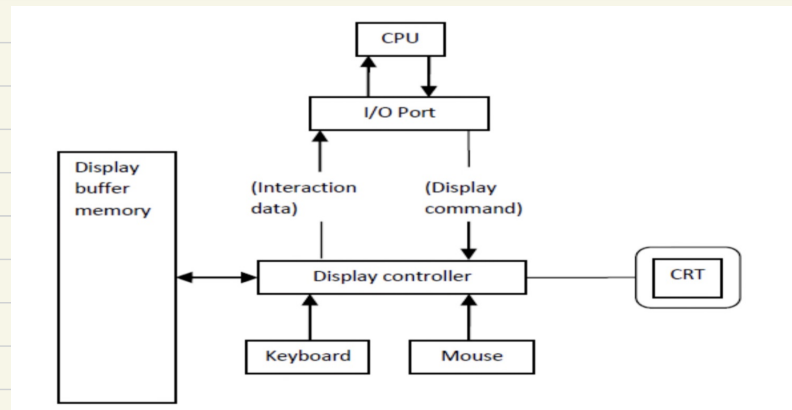
Advantages:

- Well-suited for displaying geometric shapes, diagrams, and line art.
- Can produce high-quality images with smooth curves and lines.
- Efficient for displaying complex, detailed images with minimal computational load.

Disadvantages:

- Limited in displaying continuous-tone images or photographs.
- Inefficient for filling large areas with color or shading.
- Requires more complex hardware and software to control the electron beam's movement.

→ Architecture of Vector Scan Displays :→



The architecture of a vector scan display involves several key components that work together to create images by directly drawing lines and shapes using an electron beam. Here's an explanation of the main components:

Vector Generator:

The vector generator is responsible for receiving commands to draw lines, curves, and other shapes. It generates the coordinates for the endpoints of these shapes and sends them to the display controllers. It also manages the scaling and positioning of the shapes on the screen.

Display Controllers:

The display controllers interpret the commands received from the vector generator and generate control signals for the electron beam and other components. They determine the timing and sequence of the electron beam movement to create the desired shapes on the screen.

Display Buffer:

The display buffer is a memory area that stores the instructions and coordinates generated by the vector generator. It holds the information needed to draw the lines and shapes on the screen. The display buffer acts as a temporary storage space where drawing instructions can be organized and accessed in the correct order.

② Raster Scan Displays:

In raster scan displays, the screen is divided into a grid of pixels (picture elements), and images are created by scanning the electron beam horizontally across each row of pixels, moving from top to bottom. As the beam scans each row, it turns on or off to create the desired image. This technique is commonly used in most modern displays, including computer monitors and televisions.

In raster scan displays, there are two main types of scanning methods: interlaced scanning and non-interlaced scanning. These methods determine how the electron beam moves across the screen and how the image is displayed. Let's take a look at both types:

④ Interlaced Scanning:

In interlaced scanning, the electron beam scans every other line of the screen during one pass (odd-numbered lines during the first pass), and then returns to scan the remaining lines (even-numbered lines) during the next pass. This process continues, alternating between odd and even lines, until the entire screen is covered. The result is that each frame is divided into two fields, which are displayed in rapid succession to create a complete image.

⑥ Non-Interlaced Scanning (Progressive Scanning):

In non-interlaced scanning, also known as progressive scanning, the electron beam scans each line of the screen sequentially, from top to bottom, in a single pass. This method ensures that the entire frame is displayed at once, without alternating between odd and even lines.

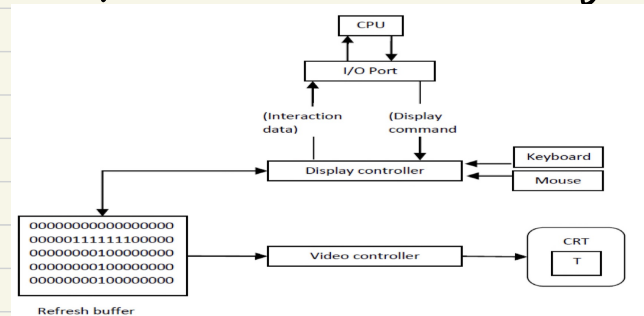
Advantages: (*raster scan*)

- Well-suited for displaying continuous-tone images, photographs, and videos.
- Efficient for filling large areas with color or shading.
- Easier to implement and control, making it suitable for a wide range of applications.

Disadvantages:

- May exhibit jagged edges and pixelation, especially at lower resolutions.
- Difficult to create smooth, high-quality lines and curves compared to vector scan displays.
- The scanning process can introduce flickering, particularly at lower refresh rates.

→ Architecture of Raster Scan Displays:→



The architecture of a raster scan display involves several components working together to generate and display images on the screen. Here's an overview of how they work:

1. **Display Processor or Controller:**

The display processor or controller is responsible for generating the visual content that needs to be displayed on the screen. It receives input data, such as images, videos, or graphics, and converts this data into a format that can be displayed on the screen. The controller also manages the overall operation of the display, including coordinating with other components and handling user interactions. It communicates with the refresh buffer to update the screen's content.

2. **Refresh Buffer:**

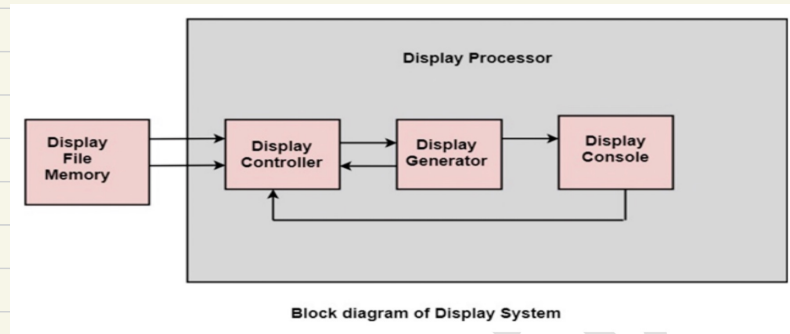
The refresh buffer is a portion of memory that stores the image data to be displayed on the screen. It holds the pixel values, color information, and other attributes of the image. The display processor updates the content of the refresh buffer based on the input data it receives. The buffer is organized in a way that corresponds to the screen's resolution and color depth.

3. **Video Controller:**

The video controller is responsible for controlling how the image data from the refresh buffer is actually displayed on the screen. It generates the timing signals that determine how the electron beam moves across the screen during the scanning process. The video controller ensures that each pixel's color value is sent to the appropriate electron beam position at the right time to create a coherent image.

The video controller manages aspects such as horizontal and vertical synchronization, pixel clock frequency, and signal timing. It ensures that the image is drawn row by row, left to right, and top to bottom, following the raster scan pattern. The controller also handles tasks like adjusting brightness, contrast, and other display parameters.

→ Display Processor :->



A display processor is a specialized component within a computer system that manages the generation and control of visual output to a display device, such as a monitor or screen. It handles tasks related to rendering graphics, text, images, and other visual elements on the screen. The display processor takes input data and transforms it into a format that can be displayed on the output device.

The main components of a display processor include:

Display File Memory:

Display file memory is a portion of the system's memory dedicated to storing the graphical data that needs to be displayed on the screen. It holds information about the location, attributes, and content of various visual elements, such as images, text, shapes, and colors.

Display Generator:

The display generator is a part of the display processor responsible for converting the graphical data from the display file memory into a format that the display device can understand. This involves generating signals that specify the color, intensity, and positioning of each pixel on the screen. The display generator also manages tasks like scaling, rotation, and other transformations.

Display Console:

The display console is the interface through which users interact with the display processor and control the visual output. It may consist of hardware controls, buttons, knobs, or a graphical user interface (GUI) that allows users to adjust settings, select display modes, and manage the content displayed on the screen.

→ Difference b/w random scan and raster scan displays:

Vector scan / Random Scan	Raster Scan
The resolution of random scan is higher in comparison raster scan.	The resolution of raster scan is lower in comparison to random scan.
It is expensive in comparison to raster scan.	It is inexpensive in comparison to random scan.
Any alterations can be done easily.	Alterations are difficult to make.
The concept of interweaving is not used.	The concept of interweaving is used.
A mathematical function is used to render an image or a picture.	To render image or picture, pixels are used.
It is suited for applications that require polygon drawings.	It is suitable to create realistic scenes.
An example of random scan is a pen plotter.	A TV set is an example of raster scan.
Random scan has lower refresh rate about 30 to 60 times per second.	Raster scan has higher refresh rate about 60 to 80 times per second.

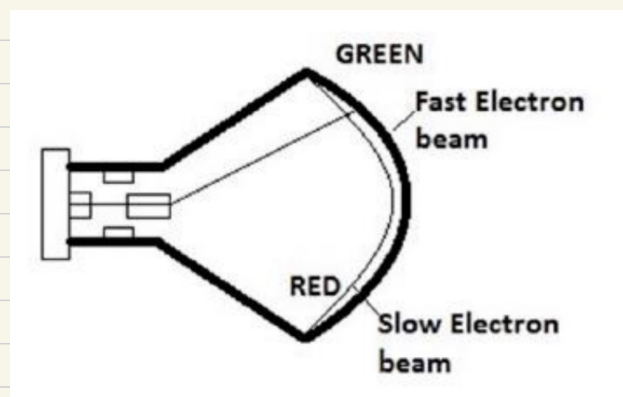
→ Color CRT Monitor :-

A color CRT (Cathode Ray Tube) monitor is a type of display device that uses a cathode ray tube to generate images in full color. It is an older technology that was widely used before the advent of modern flat-panel displays like LCDs, LEDs, and OLEDs. Color CRT monitors were commonly used for computer displays, televisions, and other applications where color reproduction was essential.

There are two basic techniques for color displays :-

1. Beam-penetration technique
2. Shadow-mask technique

→ Beam Penetration Technique :-



The beam-penetration technique, also known as beam-indexing, is a color generation technique used in color cathode ray tube (CRT) monitors. This technique involves the use of multiple electron beams to illuminate the phosphor dots on the screen, creating the illusion of full-color images. Each electron beam is associated with a different color, typically red, green, and blue (RGB).

How It Works:

In the beam-penetration technique, three electron guns are used, each emitting a focused beam of electrons. These electron beams strike the phosphor-coated screen, causing the phosphor dots to emit light. By controlling the intensity of each electron beam, the CRT can create a wide range of colors by varying the combination of red, green, and blue light. The electron beams are typically controlled by adjusting the voltages applied to the electron guns. This allows for precise control over the brightness of each color channel, enabling the display of different colors and shades.

Advantages:

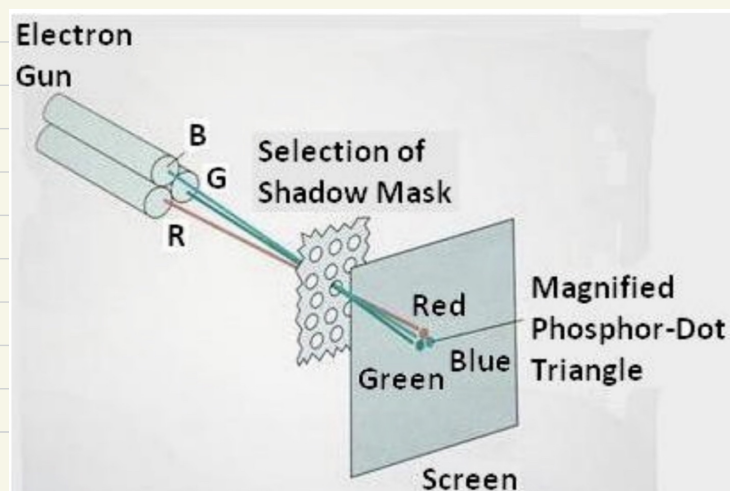
- Reduced Color Impurities
- Sharper Images
- Improved Color Reproduction

Disadvantages:

- Complex Design
- Manufacturing Challenges
- Cost
- Limited Adoption

→ Shadow-mask technique :-

The shadow mask technique is a method used in color cathode ray tube (CRT) monitors to display color images. It involves the use of a shadow mask, which is a perforated metal sheet positioned between the electron gun and the phosphor-coated screen. The shadow mask technique enables the accurate alignment of the electron beams to the appropriate phosphor dots for each color, resulting in the display of full-color images.



How It Works:

The shadow mask is a fine metal screen with thousands of tiny holes arranged in a precise pattern.

Three electron guns are used, each emitting a focused beam of electrons representing one of the primary colors: red, green, and blue.

The shadow mask is positioned a short distance behind the screen. Each hole in the mask corresponds to a specific phosphor dot on the screen.

The electron beams from the three guns pass through the holes in the mask and strike the phosphor dots on the screen. The phosphors emit light when excited by the electron beams.

The key principle behind the shadow mask technique is to ensure that each electron gun strikes only the appropriate color phosphor dots. This precise alignment prevents unwanted mixing of colors and ensures accurate color reproduction.

Advantages:

- Accurate Color Reproduction

- Bright and Vibrant Colors

- Suitable for CRTs

Disadvantages:

- Complex Manufacturing

- Color Misalignment

- Resolution Limitation

- Bulk and Weight

→ Direct View Storage Tube (DVST) :-

A directive view storage tube (DVST) is a type of cathode ray tube (CRT) used for displaying and storing images, particularly in applications such as radar and real-time data visualization. It combines the features of a CRT display with the ability to temporarily store an image for prolonged viewing. The DVST allows an image to be displayed continuously without the need for constant refreshing, making it suitable for certain specialized applications.

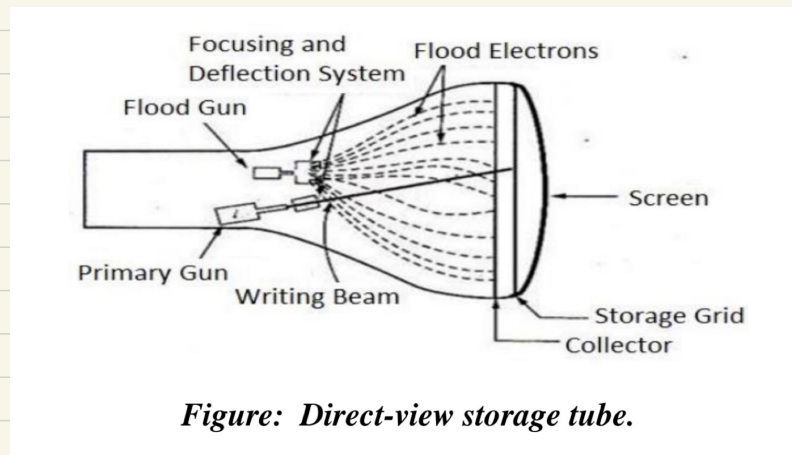


Figure: Direct-view storage tube.

The main components of a directive view storage tube include:

Electron Gun:

The electron gun emits a focused beam of electrons that strikes the phosphor-coated screen. This beam is used to create the initial image on the screen.

Phosphor-Coated Screen:

The interior surface of the CRT is coated with phosphor material. When the electron beam strikes the phosphor, it causes the phosphor to emit light, creating the visual image on the screen.

Storage Grid:

The storage grid is an additional electrode placed between the electron gun and the phosphor-coated screen. It is charged to a high positive voltage. When the electron beam strikes the storage grid, it releases secondary electrons that become trapped in a space-charge cloud near the grid. These secondary electrons remain trapped for a certain period, allowing the image to be retained on the screen even after the electron beam is turned off.

Collector:

The collector electrode is positioned behind the storage grid. It collects the secondary electrons that have been trapped near the storage grid. By applying a voltage to the collector, the trapped electrons are collected and removed, causing the stored image to fade away and preparing the screen for the next image.

How It Works:

The electron gun emits a beam that strikes the phosphor-coated screen, creating an initial image on the screen.

The storage grid is charged to a high positive voltage, and the electron beam is directed toward it. Secondary electrons are released from the storage grid and become trapped in a space-charge cloud near the grid.

The trapped secondary electrons maintain the image on the screen even after the electron beam is turned off.

To refresh the screen and remove the stored image, a voltage is applied to the collector, causing the trapped secondary electrons to be collected and removed.

The process can be repeated to display and store new images.

Advantages:

Persistence: DVSTs have the ability to retain an image on the screen for an extended period without the need for constant refreshing. This persistence is useful for applications where static images need to be displayed for an extended duration.

Low Refresh Rate: Due to their persistence, DVSTs can operate at a lower refresh rate compared to traditional CRTs, saving energy and reducing wear on the display.

Memory Effect: Images can be "written" to the screen by selectively activating specific areas of the phosphor coating. This memory effect allows for the storage of patterns or graphics.

High Contrast: DVSTs are capable of high contrast, making them suitable for applications where sharp and well-defined images are required.

Disadvantages:

Limited Applicability: DVSTs are not well-suited for dynamic or rapidly changing images, such as videos or animations, due to their persistence and memory-based operation.

Burn-In: If the same image is displayed for an extended period, the phosphor coating can suffer from burn-in or image retention, leading to a permanent ghost image on the screen.

Limited Resolution: DVSTs typically have lower resolutions compared to other CRT technologies, making them less suitable for applications requiring high levels of detail.

Complex Manufacturing: The manufacturing process for DVSTs can be more complex due to the need to control the storage and activation of specific areas of the phosphor coating.

Limited Adoption: DVST technology has been largely surpassed by more advanced display technologies, such as LCD, OLED, and modern CRTs, which offer better performance, higher resolutions, and greater versatility.

→ Flat Panel Displays :->

Flat panel displays are a type of display technology that produces visual output using a flat and thin panel, as opposed to the bulky cathode ray tube (CRT) displays. These displays are widely used in modern devices such as computer monitors, TVs, smartphones, tablets, and laptops due to their slim profile, energy efficiency, and improved image quality.

Flat panel displays are categorized into two main categories based on their underlying technology: emissive displays and non-emissive displays.

Emissive Displays:

Emissive displays generate light directly from the individual pixels or sub-pixels on the screen. Each pixel emits its own light, resulting in vibrant colors and high contrast. There are two prominent types of emissive displays: Organic Light-Emitting Diode (OLED) Displays and MicroLED Displays.

Non-Emissive Displays:

Non-emissive displays use external light sources to illuminate the screen, and the pixels manipulate the light to create images. The most common type of non-emissive display is the Liquid Crystal Display (LCD):

Some of the other commonly used Flat Panel displays are :->

LED Displays (Light-Emitting Diode Displays):

LED displays are a type of flat panel display technology that uses light-emitting diodes (LEDs) to illuminate the screen. LED displays can refer to two main types i.e., LED-Lit LCD Displays, OLED Displays

LCD Displays (Liquid Crystal Displays):

LCD displays use liquid crystal molecules to manipulate light and create images. These displays consist of a backlight that produces light and a layer of liquid crystals that control the amount of light that passes through. LCDs can be categorized into two main types: TFT-LCD Displays (Thin-Film Transistor LCD Displays), IPS-LCD Displays (In-Plane Switching LCD Displays).

Plasma Panel Displays:

Plasma panel displays, also known as plasma TVs, use a grid of cells filled with noble gases (such as neon and xenon) that become ionized to emit ultraviolet light. This ultraviolet light strikes phosphors to create visible colors. Plasma displays were popular for large-screen TVs but have largely been phased out due to advancements in LCD and OLED technology.

→ Advantages and disadvantages of flat panel displays :→

Advantages:

- Slim and lightweight
- Space-efficient and compact
- Energy efficient
- High resolution with sharp, clear images
- Flexibility in shapes and form factors
- Reduced eye strain for users
- Not susceptible to magnetic interference
- Fast response times especially with OLED

Disadvantages:

- Potential for burn-in issues especially with OLEDs
- Limited lifespan and brightness decay for OLEDs
- Backlight bleed and uniformity issues with LCDs
- Visibility challenges in bright environments for LCDs
- Motion blur due to pixel persistence in LCDs
- Complex manufacturing processes for OLED and MicroLED
- Susceptible to glare and reflections
- Color gamut limitations unable to cover full spectrum

→ Graphics Monitor :→

A graphics monitor is a type of display device specifically designed for producing and displaying high-quality visual content, such as images, graphics, and videos. Graphics monitors are optimized to accurately reproduce colors, display fine details, and provide a high level of clarity and precision. These monitors are commonly used in graphic design, video editing, animation, CAD (computer-aided design), and other visually demanding applications.

Key features of graphics monitors include:

High Resolution: Graphics monitors typically offer high resolutions to ensure that fine details are displayed accurately. This is crucial for tasks that involve precise design work.

Color Accuracy: Graphics monitors are calibrated to reproduce colors accurately, often covering a wide color gamut to ensure faithful representation of colors.

Wide Color Range: Many graphics monitors support technologies like Adobe RGB or DCI-P3 color spaces, allowing users to work with a broader range of colors.

Uniformity and Consistency: Graphics monitors strive for uniform brightness and color consistency across the entire screen to ensure accurate visual representation.

Advanced Panel Technologies: Graphics monitors often use technologies like IPS (In-Plane Switching) or OLED to provide wide viewing angles and excellent color accuracy.

Hardware Calibration: Some graphics monitors support hardware calibration, which allows users to fine-tune color accuracy using external calibration devices.

Ergonomics: Graphics monitors often come with adjustable stands, allowing users to customize the monitor's height, tilt, swivel, and rotation for comfortable viewing.

→ Workstation :→

A workstation, on the other hand, is a high-performance computer system specifically designed for tasks that require intensive processing power, such as scientific modeling, 3D rendering, computer-aided design (CAD), video editing, and software development. Workstations are optimized for stability, reliability, and efficiency in handling complex tasks.

Key features of workstations include:

- **High Processing Power:** Workstations are equipped with powerful CPUs (Central Processing Units) that can handle demanding tasks efficiently.

- **Large Amounts of RAM:** Workstations have substantial amounts of RAM to ensure smooth multitasking and efficient handling of large datasets.

- **Professional Graphics Cards:** Workstations often include professional-grade GPUs (Graphics Processing Units) optimized for tasks like 3D rendering, simulation, and scientific visualization.

- **Expandability:** Workstations allow for hardware expansion, enabling users to add more storage, additional GPUs, and other components as needed.

- **Reliability:** Workstations are designed for 24/7 operation and are built with high-quality components for long-term reliability.

- **Optimized Software Support:** Workstations come with certified drivers and software optimizations for professional applications.

- **Ergonomics and Accessibility:** Workstations may have features like tool-free access to internal components and ergonomic designs for easy maintenance.

Scan conversion:

Scan conversion, also known as rasterization, is the process of transforming continuous graphics into a collection of discrete pixels for display on a screen. It's essential in computer graphics for rendering images and shapes accurately. This process involves converting objects like lines, circles, polygons, etc., from their mathematical or geometric representations to their pixel-based representations.

Methods of Scan Conversion:

1. Analog Methods:

- Suitable for analog video systems and involves converting polar coordinate data into a raster scan data. It may use specialized hardware like vacuum tubes or delay cells for conversion.

2. Digital Methods:

- Utilizes line or frame buffers to store the image data. Various picture processing techniques are employed to read and convert this data at different speeds.

Need for Scan Conversion Algorithms:

Scan conversion accelerates the process of generating graphic objects and optimizes memory usage, improving the quality of on-screen objects.

Using these algorithms results in more efficient and faster creation of graphic objects.

It's used in systems like cathode ray tube displays, LCD monitors, projectors, and video processing techniques, serving various purposes in these applications.

Advantages of Scan Conversion:

Used in various technologies like TV displays, video capture cards, LCD monitors, projectors, and more.

Efficiently performed using high-speed integrated circuits.

Disadvantages of Scan Conversion:

During digital scan conversion, analog video signals are converted into digital data, sometimes losing quality in the process.

Can only be applied using specific integrated circuits like LSI and VLSI.

Scan Conversion of a Line in Computer Graphics:

Understanding Lines in Graphics:

A line segment connects two points and serves as a fundamental element in computer graphics. In geometry, a line is uniquely defined by two points. Algebraically, a line can be described by a slope (m) and a y-axis intercept (b). However, in computer graphics, lines are commonly defined by their two endpoints.

Line Drawing Algorithms Characteristics :

- Preserve Line Straightness: Straight lines should appear as straight lines on the display.

- Accurate Line Beginnings and Endings: Lines should accurately start and end at specified points.

- Constant Brightness: Displayed lines should have uniform brightness along their entire length, regardless of orientation or length.

- Efficiency: Drawing lines should be fast and efficient.

Methods for Line Drawing:

There are several algorithms used for line drawing in computer graphics:

- Direct Method: A simple but limited approach where the line is drawn pixel by pixel directly between two endpoints. While straightforward, it doesn't account for variations in line angles efficiently.

- Bresenham's Line Generation Algorithm: One of the most popular algorithms due to its efficiency and accuracy. Bresenham's algorithm determines which pixels to activate to draw a line between two points based on integer arithmetic operations. It works particularly well for lines with different slopes and avoids floating-point calculations, resulting in faster execution.

- Digital Differential Analyzer (DDA): Another algorithm that calculates the slope of the line and then increments the x and y coordinates at fixed intervals to approximate the line. While conceptually simple, DDA may involve floating-point arithmetic, making it slower compared to Bresenham's algorithm.

→ DDA Algorithm:→

The Digital Difference Analyzer (DDA) algorithm is used to draw lines on a screen incrementally. The algorithm is called the Digital Difference Analyzer because it interpolates points based on the difference between the start and end points. The algorithm itself is very easy to understand and implement.

Given the starting and ending coordinates of a line, DDA Algorithm attempts to generate the points between the starting and ending coordinates.

Algorithm -

Given-

Starting coordinates = (X_0, Y_0)

Ending coordinates = (X_n, Y_n)

The points generation using DDA Algorithm involves the following steps-

Step-01:

Calculate ΔX , ΔY and M from the given input.

These parameters are calculated as-

$$\Delta X = X_n - X_0$$

$$\Delta Y = Y_n - Y_0$$

$$M = \Delta Y / \Delta X$$

Step-02:

Find the number of steps or points in between the starting and ending coordinates.

if $(\text{absolute}(\Delta X) > \text{absolute}(\Delta Y))$

Steps = $\text{absolute}(\Delta X)$;

else

Steps = $\text{absolute}(\Delta Y)$;

Step-03:

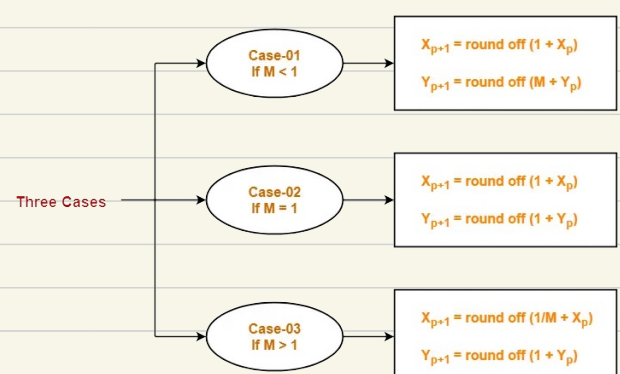
Suppose the current point is (X_p, Y_p) and the next point is (X_{p+1}, Y_{p+1}) .

Find the next point by following the below three cases-

Step-04:

Keep repeating Step-03 until the end point is reached or the number of generated new points (including the starting and ending points) equals to the steps count.

whichever value b/w ΔX and ΔY is bigger, no. of steps will be equal to that bigger value.



Problem-01:

Calculate the points between the starting point (5, 6) and ending point (8, 12).

Sol.: → Given: → $x_0 = 5, x_n = 8, y_0 = 6, y_n = 12$.

Step 1: → Calculate $\Delta x, \Delta y, M$.

$$\Delta x = x_n - x_0 = 8 - 5 = 3$$

$$\Delta y = y_n - y_0 = 12 - 6 = 6$$

$$M = \Delta y / \Delta x = 6/3 = 2$$

Step 2: →

Calculate no. of points or steps.

As $\Delta y > \Delta x$, so, no. of steps = $\Delta y = 6$.

Step 3: →

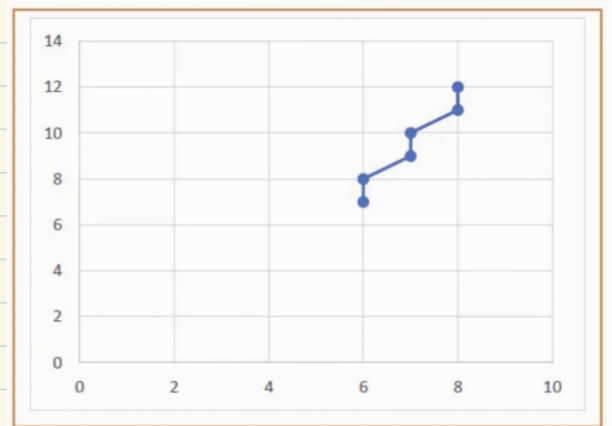
As $M > 1$, so:

$$x_{p+1} = \text{abs}(1/M + x_p) = \text{abs}(1/2 + 5) = 5.5$$

$$\text{and } y_{p+1} = \text{abs}(1 + y_p) = \text{abs}(6 + 1) = 7$$

Step 4: → Repeat the step 3 6 times as no. of steps is 6.

x_p	y_p	x_{p+1}	y_{p+1}	Round off (x_{p+1}, y_{p+1})
5	6	5.5	7	(6, 7)
		6	8	(6, 8)
		6.5	9	(7, 9)
		7	10	(7, 10)
		7.5	11	(8, 11)
		8	12	(8, 12)



Advantages of DDA Algorithm-

- 1) It is a simple algorithm.
- 2) It is easy to implement.
- 3) It avoids using the multiplication operation which is costly in terms of time complexity.

Disadvantages of DDA Algorithm-

- 1) There is an extra overhead of using round off() function.
- 2) Using round off() function increases time complexity of the algorithm.
- 3) Resulted lines are not smooth because of round off() function.
- 4) The points generated by this algorithm are not accurate.

→ Bresenham Line Algorithm :->

The Bresenham Line Algorithm is an efficient method for drawing lines on a grid-based display or calculating line-related operations. It was developed by Jack E. Bresenham in 1962 and has since become a fundamental algorithm in computer graphics and image processing.

The primary advantage of the Bresenham Line Algorithm is its ability to determine the points of a line that should be plotted to achieve the most accurate and efficient line representation. Unlike other line-drawing algorithms, such as the Digital Differential Analyzer (DDA) algorithm, which uses floating-point calculations, the Bresenham algorithm operates solely on integer arithmetic, making it faster and more suitable for implementation on resource-constrained systems.

Algorithm -

Given-

Starting coordinates = (X_0, Y_0)

Ending coordinates = (X_n, Y_n)

The points generation using Bresenham Line Drawing Algorithm involves the following steps-

Step-01:

Calculate ΔX and ΔY from the given input.

These parameters are calculated as-

$$\Delta X = X_n - X_0$$

$$\Delta Y = Y_n - Y_0$$

Step-02:

Calculate the decision parameter P_k .

It is calculated as-

$$P_k = 2\Delta Y - \Delta X$$

Step-03:

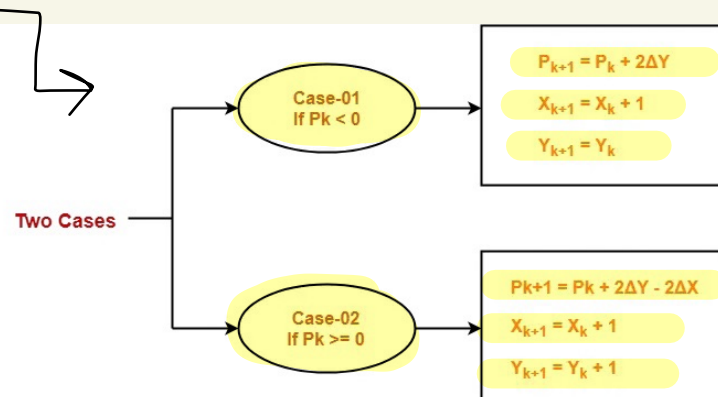
Suppose the current point is (X_k, Y_k) and the next point is (X_{k+1}, Y_{k+1}) .

Find the next point depending on the value of decision parameter P_k .

Follow the below two cases-

Step-04:

Keep repeating Step-03 until the end point is reached or number of iterations equals to $(\Delta X - 1)$ times.



Problem-01:

Calculate the points between the starting coordinates (9, 18) and ending coordinates (14, 22).

Sol.: Given $\rightarrow x_0 = 9, x_1 = 14, y_0 = 18, y_1 = 22$

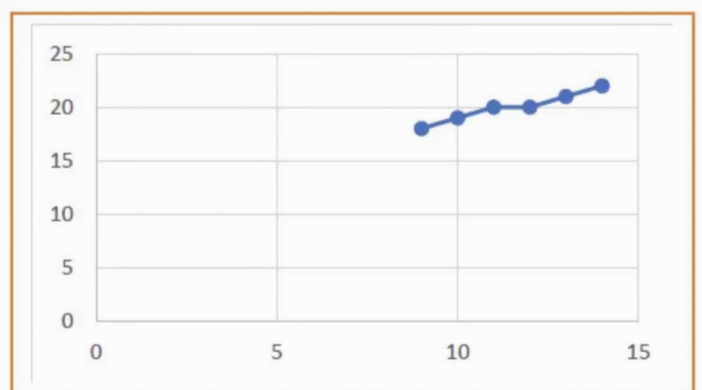
Step 1: Calculate Δx and Δy
 $\Delta x = x_n - x_0 = 14 - 9 = 5$
 $\Delta y = y_n - y_0 = 22 - 18 = 4$

Step 2: Calculate decision parameter, p_k .
 $p_k = 2\Delta y - \Delta x$
 $= (2 \times 4) - 5 = 8 - 5 = 3$

Step 3:
As $p_k \geq 0$, so $\rightarrow$
 $p_{k+1} = p_k + 2\Delta y - 2\Delta x = 3 + (2 \times 4) - (2 \times 5) = 1$
 $x_{k+1} = 1 + x_n = 1 + 9 = 10$
 $y_{k+1} = 1 + y_k = 1 + 18 = 19$

Step 4: Similarly, step 3 is repeated until the end coordinate is reached or
no. of steps = $\Delta x - 1 = 5 - 1 = 4$.

P_k	P_{k+1}	x_{k+1}	y_{k+1}
		9	18
3	1	10	19
1	-1	11	20
-1	7	12	20
7	5	13	21
5	3	14	22



Advantages of Bresenham Line Drawing Algorithm-

- 1) It is easy to implement.
- 2) It is fast and incremental.
- 3) It executes fast but less faster than DDA Algorithm.
- 4) The points generated by this algorithm are more accurate than DDA Algorithm.
- 5) It uses fixed points only.

Disadvantages of Bresenham Line Drawing Algorithm-

- 1) Though it improves the accuracy of generated points but still the resulted line is not smooth.
- 2) This algorithm is for the basic line drawing.
- 3) It can not handle diminishing jaggies.

Mid Point Circle Drawing Algorithm-

Given the centre point and radius of circle, Mid Point Circle Drawing Algorithm attempts to generate the points of one octant.

The points for other octants are generated using the eight symmetry property.

Algorithm-

Given-

Centre point of Circle = (X_0, Y_0)

Radius of Circle = R

The points generation using Mid Point Circle Drawing Algorithm involves the following steps-

Step-01:

Assign the starting point coordinates (X_0, Y_0) as-

$X_0 = 0$

$Y_0 = R$

Step-02:

Calculate the value of initial decision parameter P_0 as-

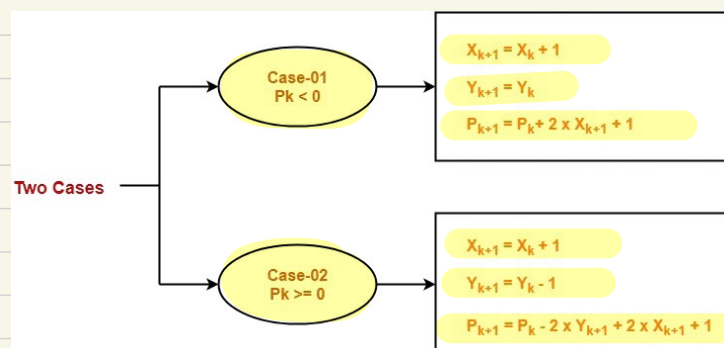
$P_0 = 1 - R$

Step-03:

Suppose the current point is (X_k, Y_k) and the next point is (X_{k+1}, Y_{k+1}) .

Find the next point of the first octant depending on the value of decision parameter P_k .

Follow the below two cases-



Step-04:

If the given centre point (X_0, Y_0) is not $(0, 0)$, then do the following and plot the point-

$X_{plot} = X_k + X_0$

$Y_{plot} = Y_k + Y_0$

Here, (X_k, Y_k) denotes the current value of X and Y coordinates.

Step-05:

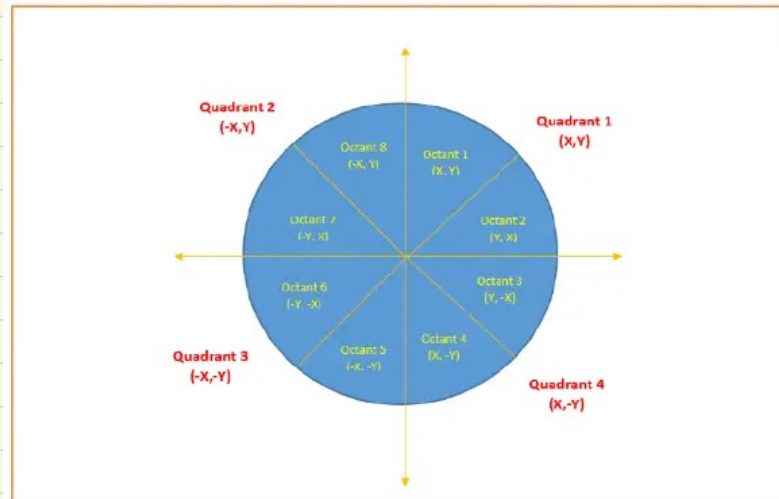
Keep repeating Step-03 and Step-04 until $X_{plot} \geq Y_{plot}$.

Step-06:

Step-05 generates all the points for one octant.

To find the points for other seven octants, follow the eight symmetry property of circle.

This is depicted by the following figure-



Problem-01:

Given the centre point coordinates $(0, 0)$ and radius as 10, generate all the points to form a circle.

Sol $\Rightarrow$ Given $\Rightarrow (x_0, y_0) = (0, 0)$, $r = 10$.

step 1 $\Rightarrow$

Take $x_0 = 0$, $y_0 = r = 10$

step 2 $\Rightarrow$ Calculate initial decision parameter, p_0 .

$$p_0 = 1 - r = 1 - 10 = -9$$

step 3 $\Rightarrow$

As $p_0 < 0$, so $\Rightarrow$

$$x_{k+1} = x_k + 1 = 0 + 1 = 1$$

$$y_{k+1} = y_k = 10$$

$$p_{k+1} = p_k + 2x_{k+1} + 1 = -9 + (2 \times 1) + 1 = -9 + 3 = -6$$

Step 4 $\rightarrow$ Since $(x_0, y_0) = (0, 0)$, so, we can't plot, if it wasn't $(0, 0)$, then

$$x_{plot} = x_k + x_0$$

$$y_{plot} = y_k + y_0$$

Step 5 $\rightarrow$ Step 3 is repeated similarly until $x_{k+1} \geq x_{k+1}$

$\swarrow$ just calculate 7 points for a quadrant

Now, the points for rest of the part are generated by following the signs of other quadrants.

The other points can also be generated by calculating each octant separately.

Here, all the points have been generated with respect to quadrant-1-

P_k	P_{k+1}	(X_{k+1}, Y_{k+1})
		(0, 10)
-9	-6	(1, 10)
-6	-1	(2, 10)
-1	6	(3, 10)
6	-3	(4, 9)
-3	8	(5, 9)
8	5	(6, 8)

Quadrant-1 (X,Y)	Quadrant-2 (-X,Y)	Quadrant-3 (-X,-Y)	Quadrant-4 (X,-Y)
(0, 10)	(0, 10)	(0, -10)	(0, -10)
(1, 10)	(-1, 10)	(-1, -10)	(1, -10)
(2, 10)	(-2, 10)	(-2, -10)	(2, -10)
(3, 10)	(-3, 10)	(-3, -10)	(3, -10)
(4, 9)	(-4, 9)	(-4, -9)	(4, -9)
(5, 9)	(-5, 9)	(-5, -9)	(5, -9)
(6, 8)	(-6, 8)	(-6, -8)	(6, -8)
(8, 6)	(-8, 6)	(-8, -6)	(8, -6)
(9, 5)	(-9, 5)	(-9, -5)	(9, -5)
(9, 4)	(-9, 4)	(-9, -4)	(9, -4)
(10, 3)	(-10, 3)	(-10, -3)	(10, -3)
(10, 2)	(-10, 2)	(-10, -2)	(10, -2)
(10, 1)	(-10, 1)	(-10, -1)	(10, -1)
(10, 0)	(-10, 0)	(-10, 0)	(10, 0)

These are all points of the Circle.

we calculated these 7 values $\leftarrow$

These 7 are swapped values of the above 7. just swap the values $\leftarrow$

→ line, parallel line, attributes, Types :→

Line:

In geometry, a line is a straight, one-dimensional figure that extends infinitely in both directions. It has no endpoints and is often represented by a straight line segment with arrows at each end to indicate its infinite length. A line is the most fundamental geometric concept and is used in various mathematical and scientific fields.

Parallel Lines:

Parallel lines are a pair of lines in the same plane that never intersect, regardless of how far they are extended in both directions. In other words, they maintain a constant distance between each other and run side by side without ever meeting. The key property of parallel lines is that they have the same slope.

Line Attributes:

Lines in geometry possess several attributes, which are important for understanding and working with them:

- Length: A line, by definition, extends infinitely in both directions and has no specific length. However, a line segment is a finite portion of a line with two endpoints, and its length can be measured.

- Direction: A line has a specific direction that can be determined by the slope of the line. The slope indicates how the line rises or falls as it extends horizontally.

- Position: A line can be positioned in various ways, including its orientation, location in a plane, and whether it intersects other lines or shapes.

- Equation: Lines can be described by linear equations in the form of $y = mx + b$, where "m" represents the slope, and "b" represents the y-intercept (the point where the line intersects the y-axis).

- Infinite Extent: Lines have infinite extent in both directions, meaning they continue indefinitely without stopping.

- Parallelism: When lines have the same slope, they are parallel and will never intersect.

- Perpendicularity: Lines that intersect at a 90-degree angle are considered perpendicular to each other. The product of the slopes of two perpendicular lines is -1.

Types of Line:

a) Curved Lines:

A line that is not straight is a curved line. If a point does not move in one direction, we get a curve.

b) Straight Lines:

The shortest line joining any 2 points is a straight line. If a point moves in only one direction, we get a straight line.

c) Horizontal Lines:

A horizontal line is a line that runs from left to right in a straight line.

Horizontal lines are parallel to the x-axis and perpendicular to the y-axis.

d) Vertical Lines:

Vertical lines run straight up and down from top to bottom or bottom to up.

Vertical lines are parallel to the y-axis and perpendicular to the x-axis.

e) Parallel Lines:

In geometry, parallel lines can be defined as two lines in the same plane that are at equal distance from each other and never meet.

f) Intersecting Lines:

When two or more lines cross each other in a plane, they are called intersecting lines. The intersecting lines share a common point, which lies on all the intersecting lines.

g) Perpendicular Lines:

In geometry, perpendicular lines are defined as two lines that meet or intersect each other at a right angle (90°).

h) Transversal Lines:

A line that intersects two or more given lines at different points is called a transversal line.

Homogeneous Coordinates:

- Used to perform translations, rotations, scaling of images in computer graphics
- Allow combining multiple transformations like translation, rotation using matrix operations
- Overcome limitation of regular coordinates where translations cannot be easily combined with other transformations
- Represent a 2D point (x, y) as a 3D homogeneous coordinate (x, y, h) where h is an arbitrary non-zero number, usually 1
- Useful for aligning, positioning and orienting images during design and construction
- Example: 2D point (3, 4) represented as homogeneous coordinate (3, 4, 1) - the extra 1 allows translating/rotating using linear algebra

Following are matrix for two-dimensional transformation in homogeneous coordinate:

1. Translation

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix} \text{ or } \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

2. Scaling

$$\begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

3. Rotation (clockwise)

$$\begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

4. Rotation (anti-clock)

$$\begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

5. Reflection against X axis

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

6. Reflection against Y axis

$$\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

7. Reflection against origin

$$\begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

8. Reflection against line Y=X

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

9. Reflection against Y= -X

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

10. Shearing in X direction

$$\begin{bmatrix} 1 & 0 & 0 \\ Sh_x & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

11. Shearing in Y direction

$$\begin{bmatrix} 1 & Sh_y & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

12. Shearing in both x and y direction

$$\begin{bmatrix} 1 & Sh_y & 0 \\ Sh_x & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Transformation in Computer Graphics

Transformation in Computer Graphics is utilized to modify the position, orientation, or size of objects within a graphical scene. By employing mathematical operations, vertices or points in a 3D or 2D space are manipulated to achieve the desired changes.

In this article, we cover Transformation in Computer Graphics explaining 2d Transformation, rotation, translation, scaling, reflection, shearing and the difference between 2d and 3d Transformation.

What is 2d Transformation in Computer Graphics?

2d Transformation in Computer Graphics is utilized to modify the position, orientation, or size of objects within a two-dimensional space. These transformations involve applying mathematical operations to the coordinates of points or vertices in order to achieve the desired changes.

These transformations can be applied in a sequence to achieve more complex effects. For example, a combination of translation, rotation, and scaling operations can be used to animate an object's movement, rotation, and resizing in a 2D animation.

The various **2d Transformation in Computer Graphics examples** include object manipulation, computer-aided design (CAD), image processing, and graphical user interfaces (GUIs). By manipulating the position, orientation, and size of objects within a 2D space, these transformations enable a wide range of visual effects and graphical applications.

The fundamental geometrical **2d Transformation in Computer Graphics** include:

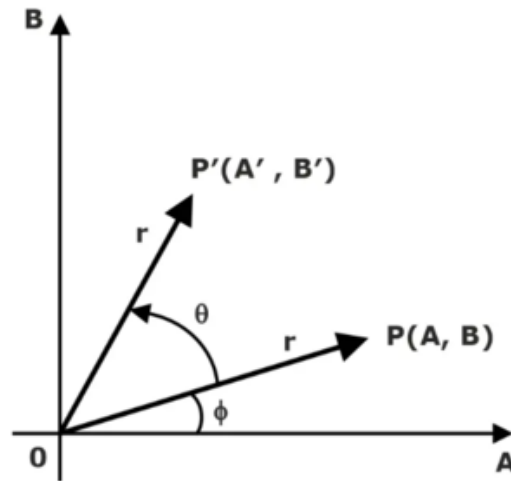
- Rotation
- Translation
- Scaling

The derived geometrical **2d Transformation in Computer Graphics** are:

- Reflection
- Shearing

2D Rotation in Computer Graphics

2D rotation is a fundamental concept that involves changing the orientation of an object or a coordinate system in a 2D plane. It enables us to rotate graphical elements around a specified point or axis by a certain angle.



To perform a 2D rotation, we need to consider two key components: the rotation angle and the rotation center. The rotation angle, denoted as θ (in radians), represents the amount of rotation to be applied. The rotation center, represented as (cx, cy) , defines the point around which the rotation will occur.

The original coordinates of points P, A, and B can be represented using standard trigonometric functions.

$$A = r \cos \phi \dots (1)$$

$$B = r \sin \phi \dots (2)$$

Similarly, the points P', A', and B' can be represented as follows:

$$A' = r\cos(\varphi+\theta) = r\cos\varphi\cos\theta - r\sin\varphi\sin\theta \dots (3)$$

$$B' = r\sin(\varphi+\theta) = r\cos\varphi\sin\theta + r\sin\varphi\cos\theta \dots (4)$$

Upon substituting equations (1) and (2) into equations (3) and (4) correspondingly, we obtain the following expressions:

$$A' = x\cos\theta - y\sin\theta$$

$$B' = x\sin\theta + y\cos\theta$$

In matrix form, the above equation can be represented as follows:

Matrix for homogeneous co-ordinate rotation (clockwise)

$$R = \begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Matrix for homogeneous co-ordinate rotation (anticlockwise)

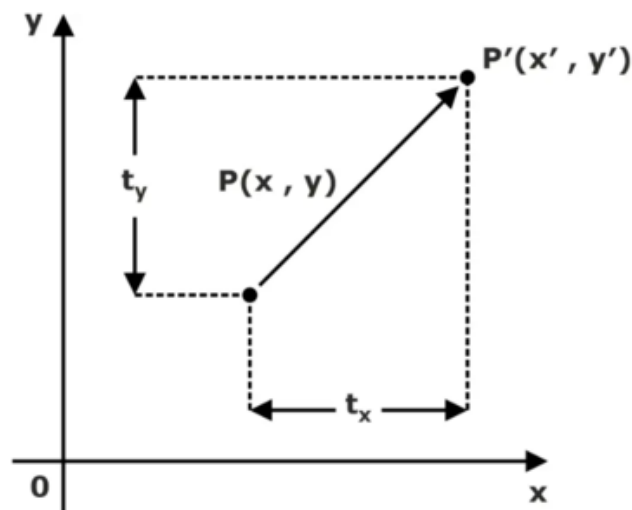
$$R = \begin{pmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Trigonometry Table: →

	0°	30°	45°	60°	90°
sin(θ)	0	$\frac{1}{2}$	$\frac{1}{\sqrt{2}}$	$\frac{\sqrt{3}}{2}$	1
cos(θ)	1	$\frac{\sqrt{3}}{2}$	$\frac{1}{\sqrt{2}}$	$\frac{1}{2}$	0
tan(θ)	0	$\frac{1}{\sqrt{3}}$	1	$\sqrt{3}$	undefined

2D Translation

2D Translation is a transformation technique that changes the position of each point in an object or a coordinate system by a specified distance in the x and y axes.



Applying 2D translation, we can say:

$$X' = X + t_x$$

$$Y' = Y + t_y$$

(t_x, t_y) represents the shift or the translation vector. The equations can be expressed using column vectors for efficient representation and computation.

$$P = [X]/[Y] \quad p' = [X']/[Y'] \quad T = [t_x]/[t_y]$$

This can also be written as:

$$P' = P + T$$

Matrix for Translation:

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix} \quad \text{Or} \quad \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

2D Scaling in Computer Graphics

2D Scaling in Computer Graphics involves resizing objects or coordinate systems in a 2D plane. It allows us to change the size of each point in the object or coordinate system by applying scaling factors in the x and y directions.

To perform 2D scaling, we utilize scaling factors: s_x for the x-axis and s_y for the y-axis. These factors determine how much each coordinate should be scaled along its respective axis.

If the scaling factor (s_x and s_y) is greater than 1, the object is enlarged and moves away from the origin. A scaling factor of 1 leaves the object unchanged, while a scaling factor less than 1 shrinks the object and moves it closer to the origin.

The equations for scaling are $X' = X * s_x$ and $Y' = Y * s_y$, where X and Y are the original coordinates of a point, and X' and Y' are the scaled coordinates after the transformation.

These equations can also be represented in matrix form as:

$$\begin{pmatrix} X' \\ Y' \end{pmatrix} = \begin{pmatrix} X \\ Y \end{pmatrix} \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \quad \text{OR} \quad \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

OR

$$P' = P \cdot S$$

2D Reflection in Computer Graphics

2D reflection is a transformation technique that involves flipping or mirroring an object or coordinate system across a specific axis in a 2D plane. It allows us to change the orientation of each point in the object or coordinate system in relation to the reflection axis.

1. Reflection about x-axis: The object can be reflected about x-axis with the help of the following matrix

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2. Reflection about y-axis: The object can be reflected about y-axis with the help of following transformation matrix

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2D Shearing in Computer Graphics

2D Shearing transformation slants or distorts an object or coordinate system along either the x-axis or y-axis in a 2D plane. It involves shifting the position of points in a specific direction based on their original coordinates.

To shear the given image along the x-axis, we use the shearing parameter sh_x .

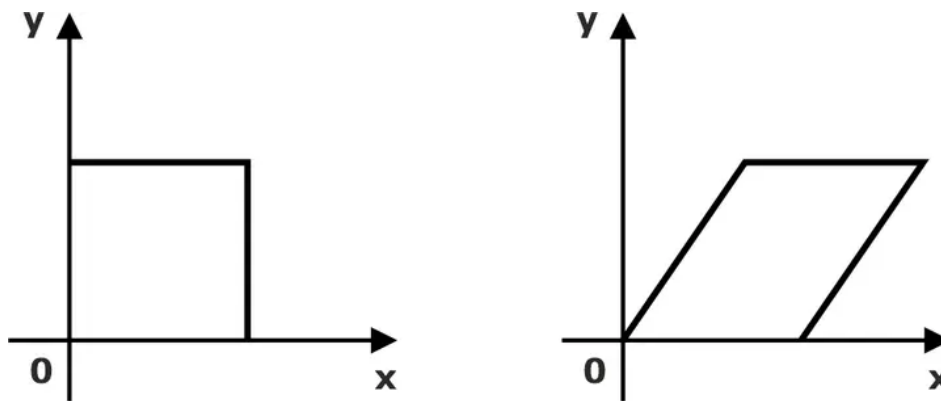


fig. shearing about the x-axis

The equation will be:

$$X' = X + Y.sh_x$$

$$Y' = Y$$

The shearing matrix along with the x-axis is:

$$\begin{bmatrix} X' \\ Y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & shx & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}$$

The shearing matrix along with the y-axis is:

To shear the provided image along the Y-axis, we utilize the shearing parameter shy.

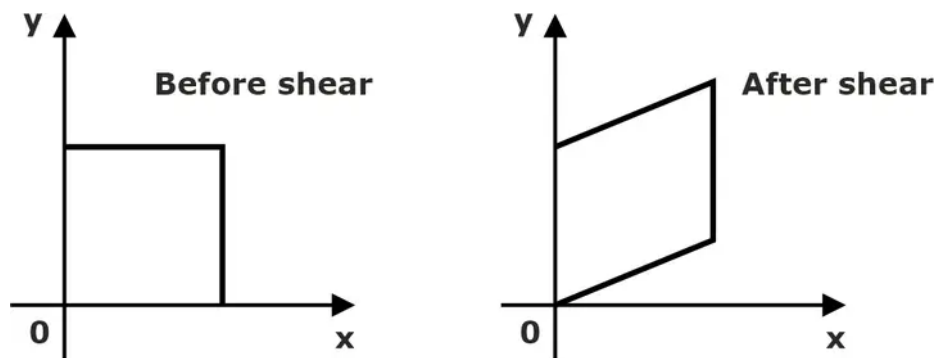


fig. shear in y direction

The equation is now:

$$Y' = X.shy + Y$$

$$X' = X$$

The shearing matrix along with the x-axis is:

$$\begin{bmatrix} X' \\ Y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ shy & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}$$

Difference between 2d and 3d Transformation in Computer Graphics

The difference between 2D and 3D transformations in computer graphics lies in the dimensionality of the space in which the transformations are applied. The prime **difference between 2d and 3d Transformation in Computer Graphics** are listed in the table below:

	2D Transformations	3D Transformations
Dimension	Two-dimensional space (x and y axes)	Three-dimensional space (x, y, and z axes)
Representation	Objects are represented on a flat surface (e.g., a computer screen)	Objects are represented in a 3D environment
Types	Translation, rotation, scaling, shearing, reflection	Translation, rotation, scaling, shearing, reflection, perspective projection, etc.
Coordinates Affected	Only x and y coordinates	x, y, and z coordinates
Depth	No depth information (z-coordinate is constant)	Depth information allows objects with volume and depth
Realism	Limited to flat, 2D representations	Enables more realistic 3D graphics and animations
Applications	GUIs, image processing, 2D animations, CAD	3D modeling, animation, virtual reality, simulations, game development.

The **Transformation in Computer Graphics** can be combined and applied in a specific order to achieve more complex effects. They serve as fundamental operations in computer graphics, enabling the manipulation of objects, creation of animations, simulation of virtual environments, and various other graphical tasks.

→ 2D transformation :-

changing the object after its creation be it size, position, etc.

↳ Diff. types of transformations :-

- ① Translation
- ② Scaling
- ③ Rotation
- ④ Reflection
- ⑤ Shearing.

↳ Translation :- shifting the object from one position to another position.

Translating a point from coordinate position (x, y) to another coordinate (x', y') , we add algebraically the translation distances T_x & T_y to original coordinates.

$$\text{So, } x' = x + T_x \\ \text{and } y' = y + T_y$$

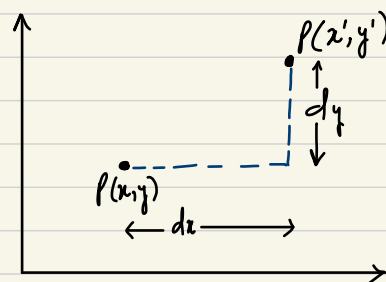
here, T_x and T_y are called shift vectors.

So, the Matrix form will be :-

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} dx \\ dy \end{bmatrix}$$

here, dx and dy are shift vectors.

$$\text{Or } P' = P + T$$



Example :- we have a $\triangle ABC$ where the coordinates are $A(2, 2)$, $B(10, 2)$ & $C(5, 5)$. Now translate the $\triangle ABC$ with $dx = 5$, $dy = 6$

Sol :-

$$A = (2, 2), B = (10, 2), C = (5, 5), dx = 5, dy = 6.$$

$$\text{Now } A' = A + T \\ = \begin{bmatrix} 2 \\ 2 \end{bmatrix} + \begin{bmatrix} 5 \\ 6 \end{bmatrix} = \begin{bmatrix} 7 \\ 8 \end{bmatrix}$$

$$\therefore A' = (7, 8)$$

$$\text{Also, } B' = B + T \\ = \begin{bmatrix} 10 \\ 2 \end{bmatrix} + \begin{bmatrix} 5 \\ 6 \end{bmatrix} = \begin{bmatrix} 15 \\ 8 \end{bmatrix}$$

$$\therefore B' = (15, 8)$$

$$\text{And, } C' = C + T \\ = \begin{bmatrix} 5 \\ 5 \end{bmatrix} + \begin{bmatrix} 5 \\ 6 \end{bmatrix} = \begin{bmatrix} 10 \\ 11 \end{bmatrix}$$

$$\therefore C' = (10, 11)$$

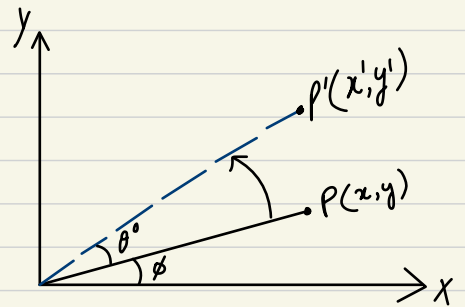
↳ Rotation $\Rightarrow$ process of changing the angle of an object. It can be clockwise or anticlockwise.

Let the coordinate of P in the polar form be $\Rightarrow$

$$x = r \cos \phi$$

$$y = r \sin \phi$$

ϕ is the angle b/w x -axis and point P .
 r is the distance from origin to P .
 θ is the angle b/w point P and point P' .



$$\text{Then, } x' = r \cos(\phi + \theta) \\ \text{and } y' = r \sin(\phi + \theta)$$

$$x' = r [\cos \phi \cos \theta - \sin \phi \sin \theta] \\ \text{or } x' = r \cos \phi \cos \theta - r \sin \phi \sin \theta \\ x' = x \cos \theta - y \sin \theta$$

$$[\because \cos(\phi + \theta) = \cos \phi \cos \theta - \sin \phi \sin \theta]$$

$$[\because x = r \cos \phi \text{ and } y = r \sin \phi]$$

$$\text{Similarly, } y' = r [\sin \phi \cos \theta + \cos \phi \sin \theta] \\ \text{or } y' = r \sin \phi \cos \theta + r \cos \phi \sin \theta \\ y' = y \cos \theta + x \sin \theta$$

$$[\because \sin(\phi + \theta) = \sin \phi \cos \theta + \cos \phi \sin \theta]$$

$$[\because x = r \cos \phi \text{ and } y = r \sin \phi]$$

Now, matrix form will be $\Rightarrow$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\text{or } P' = R \cdot P$$

The matrix form is $\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$
 and not $\begin{bmatrix} \cos \theta & -\sin \theta \\ \cos \theta & \sin \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$ because in the matrix we have to place x coordinates first then y coordinates from our eqn.

Example $\Rightarrow$ One $\triangle ABC$ has following coordinates $(2, 2), (0, 2), (5, 5)$.
 Rotate the $\triangle$ by 90° .

Sol $\Rightarrow$ here $\theta = 90^\circ$, $A = (2, 2), B = (0, 2), C = (5, 5)$

$$R = \begin{bmatrix} \cos 90^\circ & -\sin 90^\circ \\ \sin 90^\circ & \cos 90^\circ \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

Now, $A' = R.A$

$$= \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} = \begin{bmatrix} -2 \\ 2 \end{bmatrix}$$

$\therefore A' = (-2, 2)$

Also, $B' = R.B$

$$= \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 8 \\ 2 \end{bmatrix} = \begin{bmatrix} -8 \\ 2 \end{bmatrix}$$

$\therefore B' = (-8, 2)$

And, $C' = R.C$

$$= \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 5 \\ 5 \end{bmatrix} = \begin{bmatrix} -5 \\ 5 \end{bmatrix}$$

$\therefore C' = (-5, 5)$

↳ Scaling $\rightarrow$ to alter or to change the size of an object.

There are two scaling factors $\rightarrow$

$S_x \rightarrow x$ direction

$S_y \rightarrow y$ direction

Matrix form is $\rightarrow$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$P' = S.P$$

If Scaling factor = 1,

then object size will not change

If Scaling factor > 1 ,

then object size will increase

If Scaling factor < 1 ,

then object size will decrease

Example $\Rightarrow$ Make the size of the given object double, $A(2,2)$, $B(8,2)$, $C(5,5)$

Sol $\Rightarrow$ Given, $SF=2$, $A=(2,2)$, $B=(8,2)$, $C=(5,5)$

$$A' = S \times A \\ = \begin{bmatrix} 2 \\ 2 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} = \begin{bmatrix} 4 \\ 4 \end{bmatrix}$$

$$\therefore A' = (4,4)$$

$$\text{Also, } B' = S \times B \\ B' = \begin{bmatrix} 2 \\ 2 \end{bmatrix} \begin{bmatrix} 8 \\ 2 \end{bmatrix} = \begin{bmatrix} 16 \\ 4 \end{bmatrix}$$

$$\therefore B' = (16,4)$$

$$\text{And, } C' = S \times C \\ C' = \begin{bmatrix} 2 \\ 2 \end{bmatrix} \begin{bmatrix} 5 \\ 5 \end{bmatrix} = \begin{bmatrix} 10 \\ 10 \end{bmatrix}$$

$$\therefore C' = (10,10).$$

Reflection $\Rightarrow$ evaluating the mirror image of the object.

It has two types $\Rightarrow$

- 1) Reflection w.r.t x-axis.
- 2) " w.r.t y-axis.

Ref. w.r.t x $\Rightarrow$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\text{or } P' = Re(x). P$$

Ref. w.r.t y $\Rightarrow$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\text{or } P' = Re(y). P$$

Example $\Rightarrow$ A ΔABC is given, the coordinates are as $A(3,4)$, $B(6,4)$ & $C(4,8)$.
Find the reflected position of the ΔABC w.r.t x -axis.

Sol $\Rightarrow$ The matrix for $Re(x)$ is $\Rightarrow$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$A' = Re(x) \cdot A$$

$$A' = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} (3,4) = (3,-4)$$

$$\therefore A' = (3,-4)$$

$$\text{Now, } B' = Re(x) \cdot B$$

$$\text{or } B' = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} (6,4) = (6,-4)$$

$$\therefore B' = (6,-4)$$

$$\text{And } C' = Re(x) \cdot C$$

$$C' = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} (4,8) = (4,-8)$$

$$\therefore C' = (4,-8)$$

Shearing $\Rightarrow$ Changing the shape and size of an object along x and y axis.

It is similar to sliding the layers in one direction to change shape of an object.

$\rightarrow$ x -shearing $\Rightarrow$ here y -coordinates remain same.

If $P(x,y)$ is a point, new point will be $P'(x',y')$

$$x' = x + sh(x) \cdot y$$

$$y' = y$$

Matrix form $\Rightarrow$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & sh(x) \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

homogeneous coordinates $\Rightarrow$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ sh(x) & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

→ y-shearing → x-coordinates remains the same.

If $P(x, y)$ is a point, new point will be $P'(x', y')$

$$x' = x$$

$$y' = y + \sin(\theta) x$$

Matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ \sin(\theta) & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

~~Wrong~~

homogeneous coordinates:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & \sin(\theta) & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

→ 3D transformation :→

↳ 3D translation :→

Translating factors are dx, dy, dz along x, y and z axis.

We can represent it in homogeneous form like :→

$$P' = \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & dx \\ 0 & 1 & 0 & dy \\ 0 & 0 & 1 & dz \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

homogeneous factor ←

Example :→

A point has (x, y, z) coordinates in direction $(5, 6, 7)$
Translation vector in x direction

$$T_x = 3, T_y = 3, T_z = 2$$

Sol:→ Translation matrix = $\begin{bmatrix} 1 & 0 & 0 & T_x \\ 0 & 1 & 0 & T_y \\ 0 & 0 & 1 & T_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$

We know, $P' = T \cdot P$

$$\begin{aligned} \text{So } \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} &= \begin{bmatrix} 5 \\ 6 \\ 7 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 3 \\ 0 & 1 & 0 & 3 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} 8 \\ 9 \\ 9 \\ 1 \end{bmatrix} \quad \text{Or} \quad \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 9 \\ 0 & 0 & 1 & 9 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

$$\therefore x' = 8, y' = 9, z' = 9$$

What is 3D transformation?

3-D Transformation is the process of manipulating the view of a three-D object with respect to its original position by modifying its physical attributes through various methods of transformation like Translation, Scaling, Rotation, Shear, etc.

Properties of 3-D Transformation:

- Lines are preserved,
- Parallelism is preserved,
- Proportional distances are preserved.

3D transformation in homogeneous coordinates manipulates the position, orientation, and scale of 3D objects in three-dimensional space.

Homogeneous coordinates

Homogeneous coordinates extend the traditional Cartesian coordinates (X, Y, Z) with an additional coordinate (X, Y, Z, W), enabling the perspective projections using matrix multiplication.

The homogeneous coordinates (X, Y, Z, W) represents a 3D point as follows:

$$X = x \times W$$

$$Y = y \times W$$

$$Z = z \times W$$

3D transformations using homogeneous coordinates are typically represented as 4×4 matrices, known as transformation matrices.

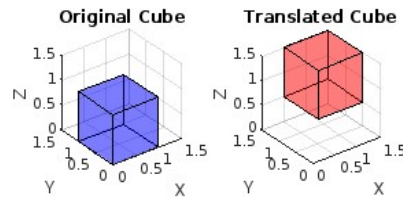
Translation

The translation moves an object in 3D space along the x, y, and z axes. Suppose $(x, y, z, 1)$ is a point in 3D space and $(x', y', z', 1)$ is a transformed point, then the translation equation would be:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Here, $(t_x, t_y, t_z, 1)$ represents the translation vector. The following figure visualizes the 3D translation:

- $X_{\text{new}} = X_{\text{old}} + T_x$ (This denotes translation towards X axis)
- $Y_{\text{new}} = Y_{\text{old}} + T_y$ (This denotes translation towards Y axis)
- $Z_{\text{new}} = Z_{\text{old}} + T_z$ (This denotes translation towards Z axis)



Blue box have been translated to the 0.5 points on x, y, and z axis.

Rotation

Rotation involves changing the orientation of an object around one or more axes. The rotation occurs along an axis. It also includes the angle of rotation θ that determines the extent to which the object will be turned about that axis. If θ is positive, the rotation would be counterclockwise.

Suppose $(x, y, z, 1)$ is a point in 3D space and $(x', y', z', 1)$ is a transformed point, then the rotation along the x-axis would be:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

For X-Axis Rotation-

This rotation is achieved by using the following rotation equations-

- $X_{\text{new}} = X_{\text{old}}$
- $Y_{\text{new}} = Y_{\text{old}} \times \cos\theta - Z_{\text{old}} \times \sin\theta$
- $Z_{\text{new}} = Y_{\text{old}} \times \sin\theta + Z_{\text{old}} \times \cos\theta$

The rotation along the y-axis would be:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

for y-axis

This rotation is achieved by using the following rotation equations-

- $X_{\text{new}} = Z_{\text{old}} \times \sin\theta + X_{\text{old}} \times \cos\theta$
- $Y_{\text{new}} = Y_{\text{old}}$
- $Z_{\text{new}} = Y_{\text{old}} \times \cos\theta - X_{\text{old}} \times \sin\theta$

The rotation along the z-axis would be:

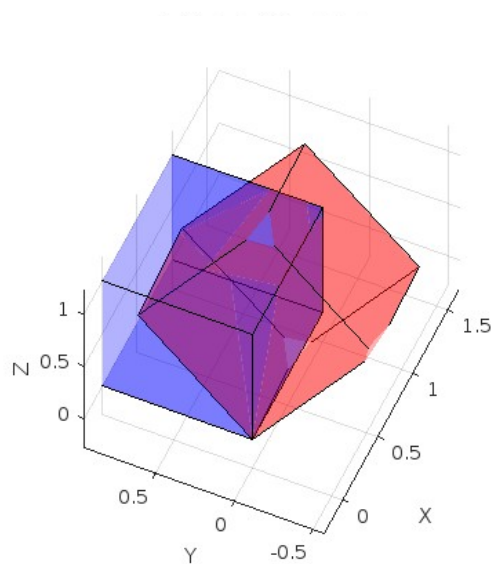
$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

For Z-Axis Rotation-

This rotation is achieved by using the following rotation equations-

- $X_{\text{new}} = X_{\text{old}} \times \cos\theta - Y_{\text{old}} \times \sin\theta$
- $Y_{\text{new}} = X_{\text{old}} \times \sin\theta + Y_{\text{old}} \times \cos\theta$
- $Z_{\text{new}} = Z_{\text{old}}$

Here, θ represents the angle of rotation. The following figure visualizes the 3D rotation:



Blue box has been rotated 30° about x-axis, 45° about y-axis and 60° about z-axis.

Scale

Scaling is the process of resizing an object in 3D space. Suppose $(x, y, z, 1)$ is a point in 3D space and $(x', y', z', 1)$ is a transformed point, then the scaling equation would be:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

where, S_x , S_y , and S_z represent the scaling factors along the x , y , and z axes, respectively. The following figure visualizes the 3D scaling:



This scaling is achieved by using the following scaling equations-

- $X_{\text{new}} = X_{\text{old}} \times S_x$
- $Y_{\text{new}} = Y_{\text{old}} \times S_y$
- $Z_{\text{new}} = Z_{\text{old}} \times S_z$

Shear

The shear transformation distorts the shape of an object along one or more axes. Suppose $(x, y, z, 1)$ is a point in 3D space and $(x', y', z', 1)$ is a transformed point, then the shearing equation would be:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & s_x & s_x & 0 \\ s_y & 1 & s_y & 0 \\ s_z & s_z & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

where s_x , s_y , and s_z are the shear factors along the x, y, and z axes, respectively. The following figure visualizes the 3D shearing:

→ The formula for shearing along YZ plane
since only you can create for other
two planes i.e., XY, XZ .

- $X_{\text{new}} = X_{\text{old}}$
- $Y_{\text{new}} = Y_{\text{old}} + Sh_y \times X_{\text{old}}$
- $Z_{\text{new}} = Z_{\text{old}} + Sh_z \times X_{\text{old}}$

Blue cube has been sheared with a factor of 0.2 along the x-axis, 0.3 along the y-axis, and 0.1 along the z-axis.

Reflection

It is also called a mirror image of an object. For this reflection axis and reflection of plane is selected.

Three-dimensional reflections are similar to two dimensions. Reflection is 180° about the given axis.

For reflection, plane is selected (xy, xz or yz).

Following matrices show reflection respect to all these three planes.

- Reflection about xy-plane (or $z = -1$)

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} X \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

- Reflection about yz-plane (or $x = -1$)

$$\begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} X \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

- Reflection about zx-plane (or $y = -1$)

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} X \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Composite Transformation

If a transformation of the plane T_1 is followed by a second plane transformation T_2 , then the result itself may be represented by a single transformation T which is the composition of T_1 and T_2 taken in that order. This is written as $T = T_1 \cdot T_2$.

Composite transformation can be achieved by concatenation of transformation matrices to obtain a combined transformation matrix.

A combined matrix –

$$[T][X] = [X] [T_1] [T_2] [T_3] [T_4] \dots [T_n]$$

Where $[T_i]$ is any combination of

- Translation
- Scaling
- Shearing
- Rotation
- Reflection

The change in the order of transformation would lead to different results, as in general matrix multiplication is not cumulative, that is $[A] \cdot [B] \neq [B] \cdot [A]$ and the order of multiplication. The basic purpose of composing transformations is to gain efficiency by applying a single composed transformation to a point, rather than applying a series of transformation, one after another.

For example, to rotate an object about an arbitrary point (X_p, Y_p) , we have to carry out three steps –

- Translate point (X_p, Y_p) to the origin.
- Rotate it about the origin.
- Finally, translate the center of rotation back where it belonged.

Example showing composite transformations:

The enlargement is with respect to center. For this following sequence of transformations will be performed and all will be combined to a single one

Step1: The object is kept at its position as in fig (a)

Step2: The object is translated so that its center coincides with the origin as in fig (b)

Step3: Scaling of an object by keeping the object at origin is done in fig (c)

Step4: Again translation is done. This second translation is called a reverse translation. It will position the object at the origin location.

Above transformation can be represented as $T_V \cdot S_T V^{-1}$

Composition of two translations:

Let t_1 t_2 t_3 t_4 are translation vectors. They are two translations P_1 and P_2 . The matrix of P_1 and P_2 given below. The P_1 and P_2 are represented using Homogeneous matrices and P will be the final transformation matrix obtained after multiplication.

$$P_1 = \begin{bmatrix} 1 & 0 & t_1 \\ 0 & 1 & t_2 \\ 0 & 0 & 1 \end{bmatrix} \quad P_2 = \begin{bmatrix} 1 & 0 & t_3 \\ 0 & 1 & t_4 \\ 0 & 0 & 1 \end{bmatrix}$$

$$P = \begin{bmatrix} 1 & 0 & t_1 + t_2 \\ 0 & 1 & t_3 + t_4 \\ 0 & 0 & 1 \end{bmatrix}$$

Above resultant matrix show that two successive translations are additive.

Composition of two Rotations: Two Rotations are also additive

Composition of two Scaling: The composition of two scaling is multiplicative. Let S_{11} and S_{12} are matrix to be multiplied.

$$S_{11} = \begin{bmatrix} S_1 & 0 & 0 \\ 0 & S_2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$S_{12} = \begin{bmatrix} S_3 & 0 & 0 \\ 0 & S_4 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$S = S_{11} * S_{12} = \begin{bmatrix} S_1 * S_3 & 0 & 0 \\ 0 & S_2 * S_4 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Unit 3: →

mapping coord. of object from wcs to dc

2D Viewing in Computer Graphics:

Two-dimensional viewing in computer graphics refers to the process of mapping the coordinates of objects from a world coordinate system (representing the actual object's coordinates in the real world) to device coordinates (representing the object's coordinates on the output screen, like a monitor).

2D Viewing Functions:

→ transf. & projecting objects from wcs to NVC and then to DC

2D viewing functions are essential tools in computer graphics for transforming and projecting objects from the world coordinate system to the normalized viewing coordinate system and then to the device coordinate system. They play a crucial role in defining the viewing perspective and ensuring that objects are displayed correctly on the screen.

Key Functions in 2D Viewing:

2 $V_{orm}()$ → WC → VC

1) The `evaluateViewOrientationMatrix()` function calculates the view transformation matrix that defines the mapping from world coordinates to viewing coordinates

→ $evmm()$ → wind. to viewport transf.

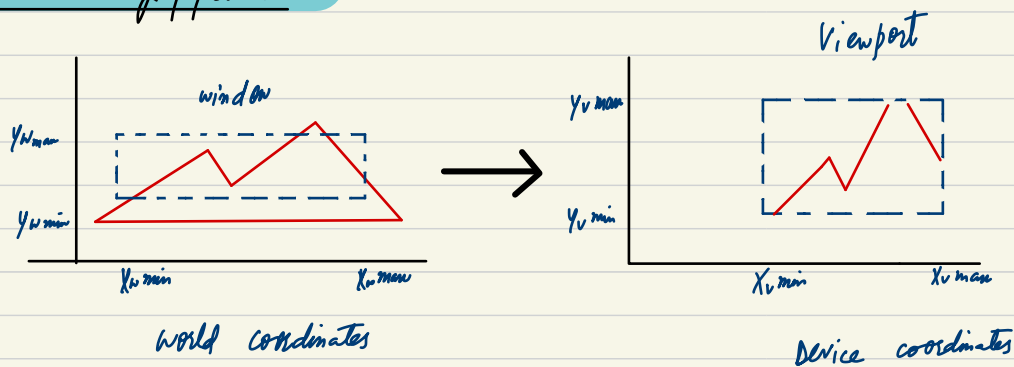
2) The `evaluateViewMappingMatrix()` function calculates the window-to-viewport transformation matrix

3) `setViewIndex()` selects a stored view by its index to be applied for rendering

4) `setWorkstationWindow()` sets output window size on specific device

5) `setWorkstationViewport()` sets output viewport on specific device

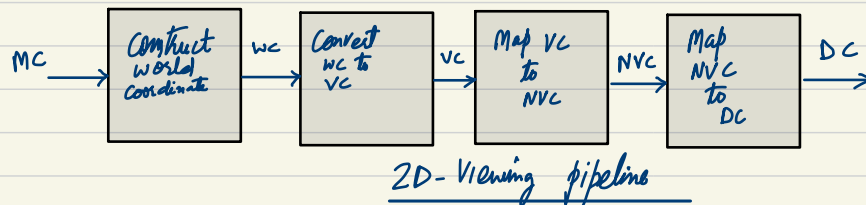
2-D Viewing pipeline



- ↳ A world coordinate area selected for display is called a window
- ↳ An area on a display device to which a window is mapped is called a viewport.

Window defines $\rightarrow$ what is to be viewed.

Viewport defines $\rightarrow$ where it is to be displayed.



The 2D viewing pipeline is a series of transformations that convert 3D objects into 2D images that can be displayed on a screen. It involves transformations between several coordinate systems:

Model (Object) coordinates:

- Coordinates used when building a model or object

World coordinates:

- Coordinate system for entire scene with all models assembled

Viewing coordinates:

- Coordinate system from point of view of camera/viewer

Normalized coordinates:

- Common plane that objects are projected onto (window)
- Device-independent

Device coordinates:

- Coordinates of specific output device after mapping from normalized space

Transformations in the pipeline:

- Model to World: Assemble models into scene
- World to Viewing: Position scene relative to viewer
- Viewing to Normalized: Project onto common window plane
- Normalized to Device: Map window to output device

Going through this viewing pipeline transforms raw geometry data into final image that can be displayed

Viewing coordinate reference frame:

The viewing coordinate reference frame is a coordinate system that is used to specify the position, orientation, and scaling of the view of a 3D scene. It is used to convert 3D world coordinates into 2D screen coordinates, which allows 3D objects to be displayed on a 2D screen.

Establishing the Viewing Coordinate Reference Frame:

The viewing coordinate reference frame is established by selecting a viewing origin (P_o) and a view up vector (V). The viewing origin is the point in the world coordinate system where the viewer is located. The view up vector is a vector that points in the direction that the viewer is looking.

Transforming World Coordinates to Viewing Coordinates:

World coordinates are transformed to viewing coordinates using a two-step transformation:

- 1 Translation: The viewing origin is translated to the world origin.
- 2 Rotation: The coordinate system is rotated so that the viewing axes align with the world axes.

The composite transformation matrix for this process is:

$$T * R$$

Where:

- T is the translation matrix
- R is the rotation matrix

The viewing coordinate reference frame is used to define a window in the world coordinate system.

Window to viewport transformation:

Window to Viewport Transformation is a vital process in computer graphics that involves mapping 2D world-coordinate objects onto device coordinates. It essentially decides how objects within a defined world or clipping window will be displayed on the viewport, which represents the screen area.

General Terms:

World coordinate: The Cartesian coordinate system in which the objects are defined.

Device coordinate: The coordinate system of the output device (e.g., screen) where the objects are to be displayed.

Window: The rectangular area of the world coordinate system that is selected for display.

Viewport: A viewport is a rectangular area on the screen where a window is displayed. It is defined in normalized coordinates, which means that it is independent of the screen resolution. The viewport can be used to display only a portion of the window, or to enlarge or shrink the window.

Window-to-viewport transformation: The process of mapping the window to the viewport.

Scaling factor: A factor that is used to adjust the size of the window or viewport.

Mathematical Calculation of Window to Viewport:

Given a point (x_w, y_w) in the window and the corresponding point (x_v, y_v) in the viewport, the transformation is calculated using scaling factors for x and y coordinates:

1. Scaling Factor Calculation:

$$\begin{aligned} \bullet \quad S_x &= \frac{X_{vmax} - X_{vmin}}{X_{wmax} - X_{wmin}} \\ \bullet \quad S_y &= \frac{Y_{vmax} - Y_{vmin}}{Y_{wmax} - Y_{wmin}} \end{aligned}$$

2. Coordinate Transformation:

$$\begin{aligned} \bullet \quad \text{For x-coordinate: } X_v &= X_{vmin} + (x_w - X_{wmin}) \times S_x \\ \bullet \quad \text{For y-coordinate: } Y_v &= Y_{vmin} + (y_w - Y_{wmin}) \times S_y \end{aligned}$$

Example Calculation:

Let's consider:

- Window: $X_{wmin} = 20, X_{wmax} = 80, Y_{wmin} = 40, Y_{wmax} = 80$.
- Viewport: $X_{vmin} = 30, X_{vmax} = 60, Y_{vmin} = 40, Y_{vmax} = 60$.
- Point in window: $(30, 80)$
- Calculate its corresponding point in the viewport.

1. Calculate Scaling Factors:

$$\begin{aligned} \bullet \quad S_x &= \frac{60 - 30}{80 - 20} = \frac{30}{60} = 0.5 \\ \bullet \quad S_y &= \frac{60 - 40}{80 - 40} = \frac{20}{40} = 0.5 \end{aligned}$$

2. Compute Point in Viewport:

$$\begin{aligned} \bullet \quad X_v &= 30 + (30 - 20) \times 0.5 = 35 \\ \bullet \quad Y_v &= 40 + (80 - 40) \times 0.5 = 60 \end{aligned}$$

The point in the window $(30, 80)$ maps to the point in the viewport $(35, 60)$. This transformation facilitates displaying objects defined within the world coordinate window onto the viewport, allowing proper positioning and scaling on the screen.

Clipping operations:

Clipping is a fundamental operation in computer graphics that involves identifying and removing portions of a graphical object that fall outside of a specified region of interest. This region is typically a rectangular window or viewport on the display device, but it can also be a more general shape such as a polygon or even a curved boundary.

Need for Clipping:

- Improves rendering performance by discarding invisible portions of objects.
- Ensures correct object appearance by preventing objects from bleeding through window boundaries.
- Enhances user interaction by allowing selective manipulation of object portions.

Applications of clipping:

- 3D viewing - remove non-visible portions of 3D models
- Antialiasing - prevent jagged lines and boundaries
- Multi-window displays - show independent windows
- Drawing/painting - select, copy, move parts of images
- User interaction - facilitate manipulations of objects

Point clipping:

Point clipping is a fundamental algorithm in computer graphics that determines whether a point lies inside or outside a specified region or boundary. It is particularly useful when dealing with objects that have complex shapes or when displaying only a portion of an image.

Clipping Algorithm:

The point clipping algorithm is a simple and efficient method for determining whether a point lies inside a rectangular window. The algorithm works by checking the point's coordinates against the boundaries of the window. If any of the coordinates fall outside the window, then the point is considered to be outside the window and is discarded.

Conditions for Point Clipping:

The point (x, y) is checked against four conditions based on the window's boundaries:

$$\begin{aligned} x \leq x_{\max} &\rightarrow \text{is } x \text{ less than or equal to } x_{\max} \\ x \geq x_{\min} &\rightarrow \text{is } x \text{ greater than or equal to } x_{\min} \\ y \leq y_{\max} &\rightarrow \text{is } y \text{ less than or equal to } y_{\max} \\ y \geq y_{\min} &\rightarrow \text{is } y \text{ greater than or equal to } y_{\min} \end{aligned}$$

Applications of Point Clipping:

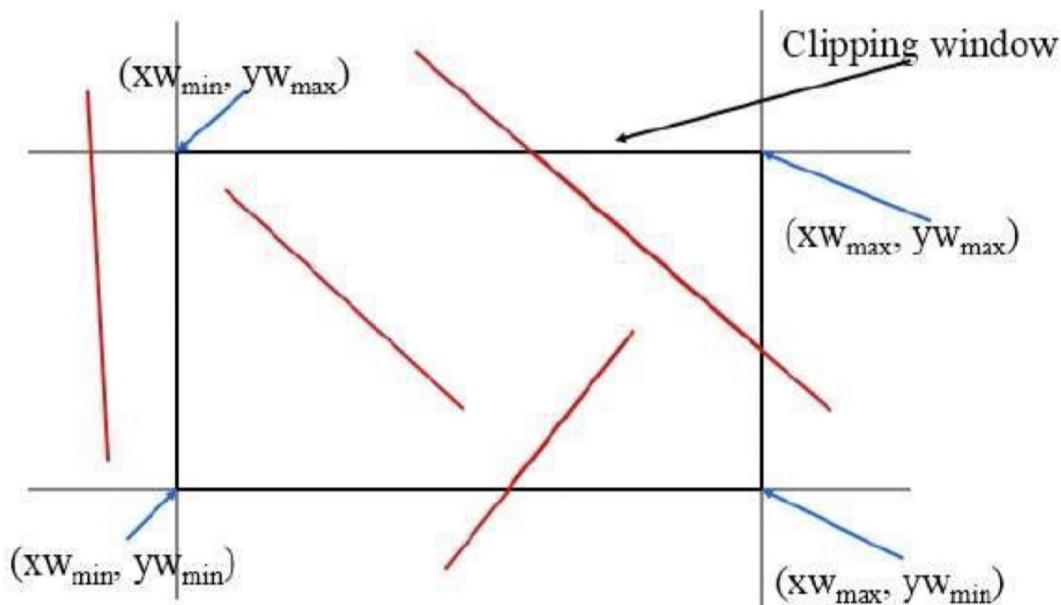
- Line clipping: Point clipping is used as a preliminary step in line clipping algorithms.
- Polygon clipping: Point clipping can be used to determine which points of a polygon lie inside a window, which is useful for polygon filling algorithms.
- Image processing: Point clipping can be used to extract a portion of an image.
- User interaction: Point clipping can be used to determine whether a user has clicked inside or outside of a specific area of the screen.

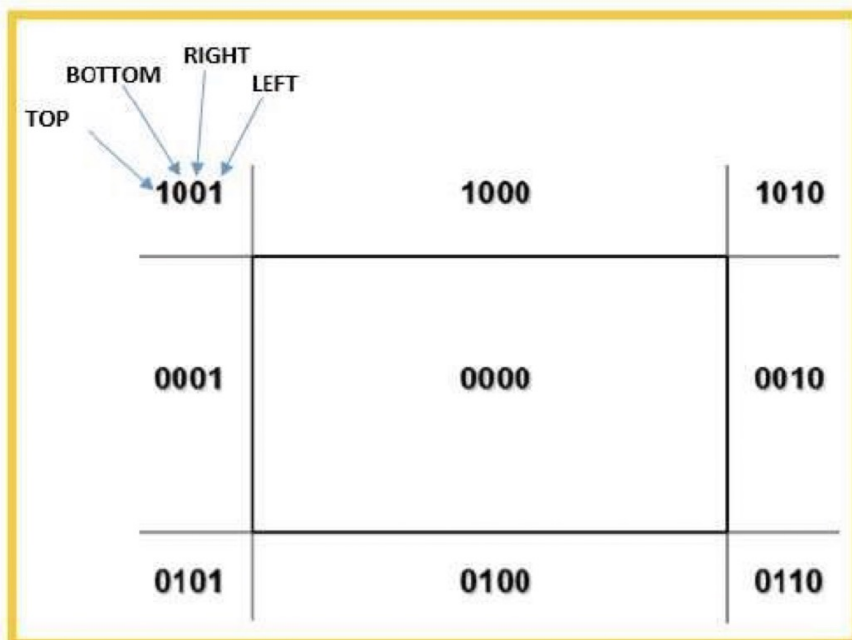
Line Clipping:

Line clipping is a fundamental operation in computer graphics that involves removing portions of line segments that fall outside a specified region of interest. This region is typically a rectangular window or viewport on the display device, but it can also be a more general shape such as a polygon or even a curved boundary. Clipping line segments ensures that only the visible portions of lines are displayed, which can improve rendering performance.

Cohen-Sutherland line clipping:

The Cohen-Sutherland line clipping algorithm is a fast and efficient method for clipping line segments against a rectangular window. It is based on the idea of dividing the window into nine regions and then determining which region each endpoint of the line falls into. Based on the regions of the endpoints, the algorithm can determine whether the line is completely inside, completely outside, or partially inside the window. If the line is completely inside or completely outside, no further processing is required. However, if the line is partially inside the window, the algorithm calculates the intersection points of the line with the window boundaries and then discards the portions of the line that fall outside the window.





Algorithm:

1 Region Code Assignment: Assign a 4-bit region code to each endpoint of the line, representing its position relative to the clipping window:

- Top (T)
- Bottom (B)
- Right (R)
- Left (L)

2 Trivial Acceptance: If both endpoints have a region code of 0000 (inside the window), accept the line without modification.

3 Trivial Rejection: If the logical AND of the region codes is not 0000 (completely outside the window), reject the line.

4 Clipping: If neither of the above conditions is met, proceed with clipping:

- Select the endpoint outside the clipping window.
- Determine the intersection point between the line and the clipping window boundary based on the endpoint's region code.
- Replace the selected endpoint with the intersection point and update its region code.
- Repeat steps 2 or 4 until the line is trivially accepted or rejected.

5 Iteration: Repeat steps 1-4 for all remaining lines.

Line clipping using non rectangular clip windows:

Line clipping involves removing portions of line segments that lie outside a specified region of interest. This region can be a rectangular window, viewport, or a more general shape like a polygon or curved boundary. Clipping against non-rectangular clip windows is more complex but essential for many applications.

- More complex than clipping against rectangular viewports
- Uses parametric line equations (Liang-Barsky, Cyrus-Beck)
- Can handle convex and concave polygonal clip shapes

Key Steps:

Preliminary screening:

- Check if line completely outside clip polygon
- Split concave polygons into convex parts

Clipping against curved boundaries:

- Clip against bounding rectangle first
- Test distance against circle center
- Compute circle-line intersections

Splitting concave polygons:

- Check sign of z-component of edge cross products
- Split along first edge vector when sign flips
- Deal with degenerate cases

Polygon clipping:

It involves removing portions of a polygon that fall outside a specified region of interest. This region is typically a rectangular window or viewport on the display device, but it can also be a more general shape such as a polygon or even a curved boundary. Clipping polygons prevents polygons from bleeding through the boundaries of the window or viewport and for ensuring that only the visible portions of polygons are displayed.

Sutherland-Hodgeman Polygon Clipping:

It is a widely used and efficient method for clipping polygons against a rectangular window. It is a divide-and-conquer algorithm that works by processing the polygon's edges one at a time against each boundary of the window. The algorithm maintains a list of vertices that represent the visible portion of the polygon, and it updates the list as it processes each edge.

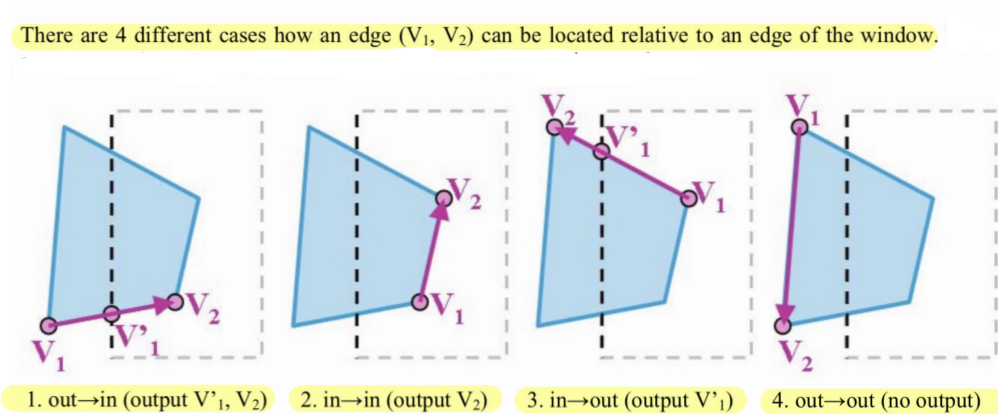
There are four possible cases that can occur when clipping a polygon edge against a window boundary:

- 1 Trivial acceptance: Both vertices of the edge lie inside the window. In this case, the 2nd vertex is added to the output list.
- 2 Trivial rejection: Both vertices of the edge lie outside the window. In this case, no part of the edge(no vertices) is added to the output list.
- 3 Outside-to-inside: The first vertex lies outside the window, and the second vertex lies inside the window. In this case, the intersection point between the edge and the window boundary is added to the output list, along with the second vertex.
- 4 Inside-to-outside: The first vertex lies inside the window, and the second vertex lies outside the window. In this case, only the intersection point between the edge and the window boundary is added to the output list.

Algorithm:

The algorithm goes through a sequence of steps:

1. The algorithm begins by taking the coordinates of all vertices of the polygon and the clipping window.
2. It examines each edge of the polygon against the edges of the clipping window individually.
3. Based on the relationship between the polygon edge and the clipping boundary, it saves the resulting intersections and vertices in a new list of vertices. Four potential cases dictate this relationship:



4. This process is repeated for each edge of the clipping window. The resulting list of vertices after each iteration is passed on to process the next edge of the clipping window.

Advantages of Sutherland-Hodgeman Polygon Clipping:

Simple and efficient. It can handle polygons with any number of edges, and it can clip polygons against any rectangular window.

Drawbacks of Sutherland-Hodgeman Polygon Clipping:

it generates a large number of intermediate vertices, which can increase memory usage. Additionally, the algorithm is not as efficient for clipping polygons against non-rectangular windows.

Polygon Filling Algorithms:

Polygon filling is a fundamental operation in computer graphics that involves coloring the interior of a polygon. This is typically done by identifying the pixels that fall inside the polygon and setting their colors to the desired fill color.

There are several different polygon filling algorithms:

Scan Line Algorithm:

The scan line algorithm is one of the most common polygon filling algorithms. It works by intersecting each scan line of the display with the edges of the polygon, and then filling in the pixels between the intersection points. This algorithm is relatively simple to implement and is efficient for filling polygons with a small number of edges. However, it can be less efficient for filling polygons with a large number of edges, as it requires a large number of intersection calculations.

This algorithm works by intersecting scanline with polygon edges and fills the polygon between pairs of intersections. The following steps depict how this algorithm works:

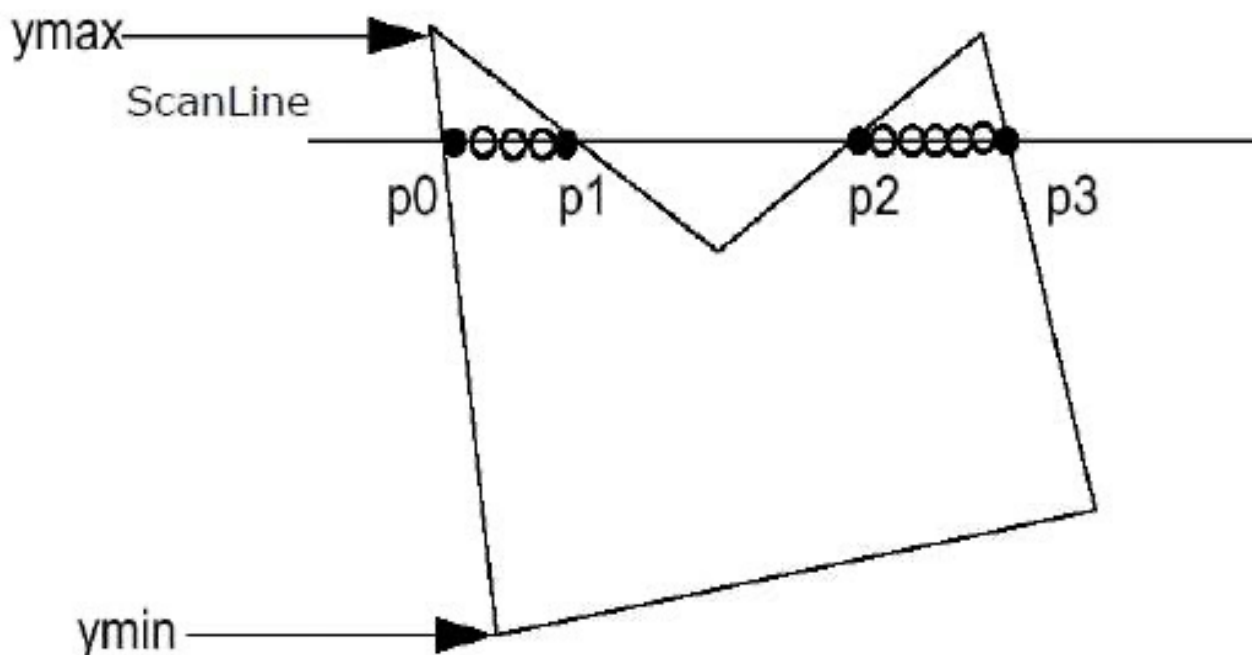
Step 1: Find out the Ymin and Ymax from the given polygon.

Step 2: ScanLine intersects with each edge of the polygon from Ymin to Ymax.

Name each intersection point of the polygon. As per the figure shown below, they are named as p0, p1, p2, p3.

Step 3: Sort the intersection point in the increasing order of X coordinate i.e. (p0, p1), (p1, p2), and (p2, p3).

Step 4: Fill all those pair of coordinates that are inside polygons and ignore the alternate pairs



Flood Fill Algorithm:

The flood fill algorithm is another common polygon filling algorithm. It works by recursively filling in the pixels that are adjacent to a seed point, which is a pixel that is known to be inside the polygon. This algorithm is efficient for filling in small, connected regions, but it can be less efficient for filling in large, non-connected regions.

used when the boundary pixels have different colors.

Algorithm $\rightarrow$

coordinates
new color
old color

```
Flood(x, y, n, o)
{
  If (getpixel(x, y) == o)
  {
    putpixel(x, y, n)
    Flood(x+1, y, n, o)
    Flood(x, y+1, n, o)
    // (x-1, y, n, o)
    // (x, y-1, n, o)
    // (x-1, y-1, n, o)
    // (x-1, y+1, n, o)
    // (x+1, y-1, n, o)
    // (x+1, y+1, n, o)
  }
}
```

$x-1, y+1$	$x, y+1$	$x+1, y+1$
$x-1, y$	x, y	$x+1, y$
$x-1, y-1$	$x, y-1$	$x+1, y-1$

$\rightarrow$ seed point

Boundary Fill Algorithm:

The boundary fill algorithm is a variation of the flood fill algorithm that works by filling in the pixels that are adjacent to the boundary of the polygon. This algorithm is efficient for filling in polygons with a well-defined boundary, but it can be less efficient for filling in polygons with holes or other irregularities.

Implemented using 4-connected or 8-connected pixels, checking adjacent pixels until encountering a boundary with a different color.

Algorithm $\Rightarrow$

```
boundary(x, y, F, b)
{
  if (getpixel(x, y) != b and getpixel(x, y) != F)
  {
```

```
    putpixel(x, y, F)
```

```
    boundary(x+1, y, F, b)
```

```
    boundary(x, y+1, F, b)
```

```
    boundary(x-1, y, F, b)
```

```
    boundary(x, y-1, F, b)
```

```
    // (x-1, y-1, F, b)
```

```
    // (x-1, y+1, F, b)
```

```
    // (x+1, y-1, F, b)
```

```
    // (x+1, y+1, F, b) }
```

$x-1, y+1$	$x, y+1$	$x+1, y+1$
$x-1, y$	x, y	$x+1, y$
$x-1, y-1$	$x, y-1$	$x+1, y-1$

here, x, y are coordinates.

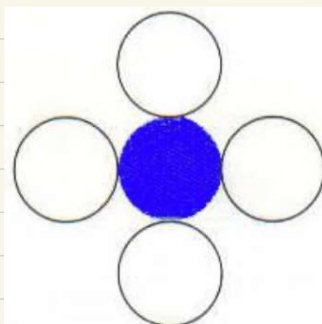
F is the color of the pixel

b is the // // // boundary pixel

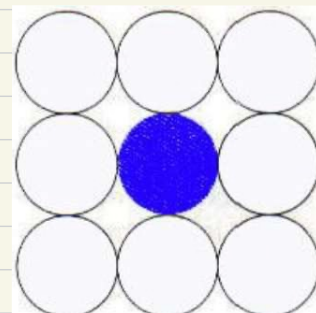
getpixel returns the color of a pixel

put pixel colors the pixel.

Boundary fill Algo is used when there is a single color of boundary pixels.



4-connected



8-connected

Inside-Outside Test:

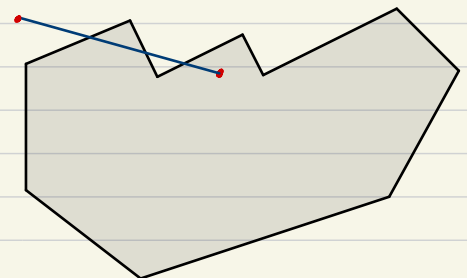
The inside-outside test is a method for determining whether a particular point lies inside or outside of a polygon. There are two common inside-outside tests: the odd-even rule and the nonzero winding number rule.

1) Odd-Even Rule:

The odd-even rule works by counting the number of times that a line from the given point to infinity crosses an edge of the polygon. If the number of crossings is odd, then the point is inside the polygon. Otherwise, the point is outside the polygon.

Algo.:

- 1) Construct a line segment s/w given point to infinity.
- 2) Count the no. of intersections.
- 3) If no. of intersections is odd, then the point is inside, else outside

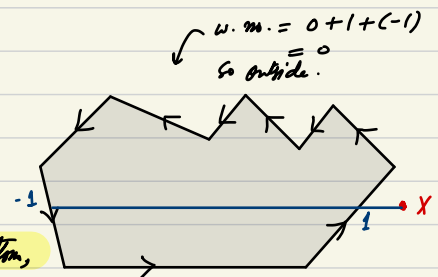


2) Nonzero Winding Number Rule:

The nonzero winding number rule works by assigning a direction to each edge of the polygon (counter clockwise direction). Then, the winding number of a point is calculated by counting the number of times that a line from the point to infinity crosses an edge of the polygon, taking into account the direction of the edge. If the winding number is nonzero, then the point is inside the polygon. Otherwise, the point is outside the polygon.

Algo.:

- (w.no.)
- 1) initialise Winding no. = 0.
 - 2) if line crosses an edge directed bottom to top, then add 1 to w.no., elif it crosses top to bottom, then add -1 to w.no.
Also, if line crosses an edge which is directed left to right or right to left, add no value.
 - 3) point outside if w.no. = 0
else inside



outside
Test

inside
test

→ Projection :→ converting 3D objects into 2D objects and transforming the object into the projection plane (view plane).

When geometric objects are formed by the intersection of lines with the plane, the plane is called projection plane and the lines are called projectors.

The arbitrary point from where the lines are drawn on each point of an object is called centre of projection.

Projections are broadly classified into 2 types :→

↳ Parallel projection.

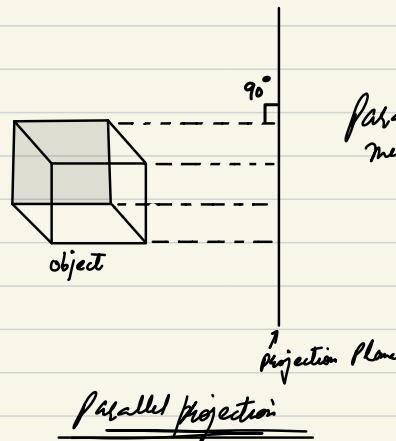
↳ Perspective projection.

Parallel projection:

Parallel projection in computer graphics is a method used to display images in their true shape and size. It involves extending parallel lines from each vertex on the object until they intersect the plane of the screen, allowing the creation of a projection where points of the object project onto the view plane.

Parallel projection is favored by architects and engineers for creating working drawings that require accurate representations of objects. It's particularly useful for generating multiple views of an object from different planes, aiding in comprehensive visualizations.

↳ If COP is located at infinity & all the lines are parallel, the result is a parallel projection.



Parallel projectors
meet at infinity. i.e.,
COP at infinity

Key Characteristics:

→ because COP is at infinity.

Instead of a center of projection, parallel projection specifies a direction of projection.

Extending the lines from vertices of the object until they intersect the view plane determines the projection of each vertex.

The distance from the center of projection to the project plane is infinite.

Parallel projections sacrifice realism for precision.

Types of Parallel Projections:

Isometric Projection: All projectors make equal angles, usually set at 30° . This projection method creates an illusion of depth without distorting the object's shape.

Dimetric Projection: Two projectors have equal angles w.r.t two principal axes.

Trimetric Projection: The direction of projection makes unequal angles with the principal axes.

Cavalier Projection: All projection lines perpendicular to the plane are projected without any change in length.

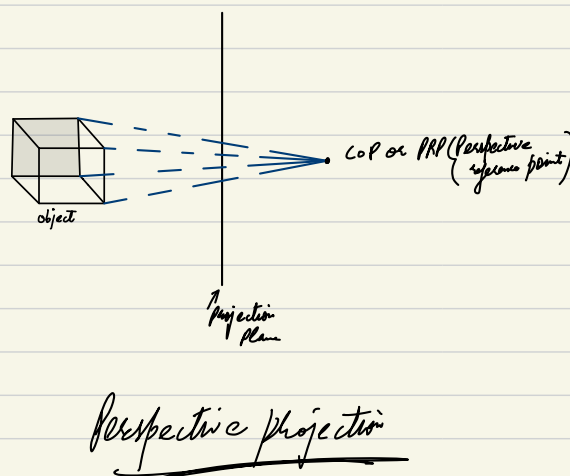
Cabinet Projection: Perpendicular lines are projected to half of their length, providing a realistic appearance while maintaining a measured approach.

Orthographic Projection: In orthographic projection the direction of projection is normal to the projection of the plane. There are three types of orthographic projections

- Front Projection
- Top Projection
- Side Projection

Perspective Projection in Computer Graphics:

Perspective projection is a method of projecting three-dimensional objects onto a two-dimensional surface. It is a common technique used to create realistic images of objects. Perspective projection mimics the way objects appear to the human eye. This is because perspective projection takes into account the effects of distance.



Properties of Perspective Projection:

- Perspective projection preserves the relative depth of objects. This creates a sense of depth in the image which is similar to how humans perceive depth in the real world.
- Perspective projection produces images that are more realistic than those produced by parallel projection. This is because it takes into account the effects of distance and foreshortening, which are two important factors in how humans perceive depth.
- Perspective projection is more complex to implement than parallel projection. This is because it requires more calculations to project objects onto the plane.

Types of Perspective Projection:

There are several different types of perspective projection, each with its own unique properties. Three common types are:

- **One-point perspective:** In one-point perspective, all the lines converge at a single point on the horizon, called the vanishing point. This type of perspective is often used for architectural drawings and illustrations.
- **Two-point perspective:** In two-point perspective, there are two vanishing points, one for each of the horizontal axes. This type of perspective is more common than one-point perspective and is often used for photographs and paintings.
- **Three-point perspective:** In three-point perspective, there are three vanishing points, one for each of the axes. This type of perspective is the most realistic, but it is also the most difficult to implement.

Difference Between Parallel and Perspective Projection in Computer Graphics

Here is a list of the differences between Parallel and Perspective Projection in Computer Graphics.

Orthographic Projection:

In orthographic projection the direction of projection is normal to the projection of the plane. There are three types of orthographic projections –

- Front Projection
- Top Projection
- Side Projection

Oblique Projection:

In oblique projection, the direction of projection is not normal to the projection of plane.

In oblique projection, we can view the object better than orthographic projection. There are two types of oblique projections – Cavalier and Cabinet.

Parallel Projection	Perspective Projection
It represents any given object in a different way- as we view it on a telescope.	It represents any given object in a three-dimensional manner.
It does not alter the shape or the size of the given object on a plane.	It alters the size or shape of the object
The distance of the given object is infinite from the center of the projection.	The distance of the given object is finite from the center of the projection.
It can provide a user with an accurate view of the given object.	It cannot provide a user with an accurate view of the given object.
projection lines are parallel to each other.	projection lines are not parallel to each other.
The projector is also parallel.	The projector is not at all parallel.
There are basically two types of parallel projections: • Oblique • Orthographic	There are basically three types of perspective projections: • One Point • Two Point • Three Point
Parallel Projection does not form a realistic view of the world and its objects.	Perspective Projection generates a very realistic view of the world and the objects present in it.

Shading:

Shading is a technique that involves adding detail and depth to the appearance of 3D objects by varying the intensity of light across their surfaces. This process is essential for creating realistic images

Importance of Shading:

Shading plays a crucial role in enhancing the realism and visual appeal of computer-generated imagery. Shading allows objects to appear more solid, textured, and three-dimensional. Without shading, 3D objects would appear flat, unrealistic, and lacking in depth.

Shading Modes:

- Wireframe mode displays only the edges of an object, leaving the faces unfilled.
- Solid shading mode fills in the faces of an object with a single color.
- Textured shading mode applies a texture to the surface of an object to add detail and realism.
- Bump mapping is a technique that simulates the appearance of surface texture without actually adding geometry to the object.
- Normal mapping is a more advanced technique for simulating surface texture that takes into account the orientation of the surface normal at each point.

Shadows:

Shadows are another important aspect of creating realistic images in computer graphics. Shadows are formed when an object obstructs the path of light, creating a region of darkness behind the object. Shadows provide additional depth cues and make objects appear more grounded in the scene.

Shadow Generation:

There are several techniques for generating shadows in computer graphics. One common method is shadow mapping, which uses a depth map to determine which parts of the scene are in shadow.

Another method is ray tracing, which casts rays of light from the light source to each pixel in the scene to determine whether or not it is in shadow.